Engineering Research
Express

INCLUSIVE PUBLISHING
TRUSTED SCIENCE

PURPOSE-LED
PUBLISHING™

PAPER

Quantum complex language processing: a framework for semantic encoding in hilbert space

To cite this article: Aadrian Routh *et al* 2026 *Eng. Res. Express* **8** 095242

View the article online for updates and enhancements.

You may also like

- Autler–Townes splitting in Rydberg atoms: transition dipole matrix element extraction and field efficiency analysis
Brian C Holloway, Gavin M Chase, Lee E Harrell et al.
- ICRH modelling of DTT in full power and reduced-field plasma scenarios using full wave codes
A Cardinali, C Castaldo, F Napoli et al.
- XFEL imaging techniques for high energy density and inertial fusion energy research at HED-HiBEF
Alejandro Laso Garcia, Mikhail Mishchenko, Victorien Bouffetier et al.

This content was downloaded from IP address 193.205.162.73 on 02/07/2026 at 15:25

Engineering Research Express

Check for updates

PAPER

RECEIVED
19 November 2025

REVISED
9 April 2026

ACCEPTED FOR PUBLICATION
28 April 2026

PUBLISHED
11 May 2026

Quantum complex language processing: a framework for semantic encoding in hilbert space

Aadrian Routh, Tapan Kumar Dey and Onkar Singh*

Department of Computer Science and Engineering, Manipal University Jaipur, Jaipur, India
* Author to whom any correspondence should be addressed.

E-mail: onkar.singh@jaipur.manipal.edu, aadrian.229301454@muj.manipal.edu and tapan.dey@jaipur.manipal.edu

Keywords: quantum complex language processing (QCLP), complex embeddings, amplitude encoding, parameterized quantum circuits, hybrid quantum–classical optimization, binary text classification

Abstract

Quantum complex language processing (QCLP) examines whether complex-valued token embeddings, when normalized into quantum states and processed by a variational circuit, can serve as a compact feature map for binary text classification. We evaluate QCLP on a binary subset of 20 Newsgroups and compare it against a real-valued quantum natural language processing baseline and a small classical neural network. QCLP consistently improves over the shallow quantum baseline in accuracy, $F1$ score, and calibration, indicating that complex-valued representations can strengthen a constrained quantum pipeline. However, the classical model remains stronger on this benchmark, so the results do not demonstrate quantum advantage. Instead, they show that the proposed quantum feature map changes the inductive bias of a compact classifier in a reproducible way. Gradient-variance analysis further indicates that optimization difficulty remains a central bottleneck for deeper variational circuits. The main contribution of this work is therefore a controlled testbed for studying complex-valued quantum text representations, together with a careful analysis of where such representations help, where they do not, and why state-preparation cost remains a practical barrier.

1. Introduction

Natural language processing (NLP) has undergone a remarkable transformation over the past decade, driven by sophisticated classical architectures and ever-expanding computational resources. Over the past decade, architectures such as recurrent neural networks, convolutional neural networks, and most prominently transformer models (BERT and GPT) have driven major gains in both language understanding and generation. Those gains rely on dense, real-valued token representations (Word2Vec, GloVe, fastText) and on contextualized embeddings (ELMo, RoBERTa [1], XLNet [2]) that adapt token vectors to surrounding text. Although classical NLP methods have achieved impressive results, they still face several practical challenges, including the efficient modeling of very long-range dependencies, the handling of rapidly growing vocabularies and subword vocabularies, and the steep increase in computation and memory requirements as model size reaches the billion-parameter scale.

To investigate an alternative representation, we consider a hybrid quantum–classical approach based on complex amplitudes. Recent progress in quantum computing has brought it from a purely theoretical idea into experimental use on noisy intermediate-scale quantum (NISQ) hardware [3], where complex-valued state vectors and entanglement provide a natural way to represent phase information in a compact form. In this work we leverage those primitives: our quantum complex language processing (QCLP) pipeline learns complex token embeddings, aggregates them into mean-pooled sentence states, and amplitude-encodes those states for a 4-qubit parameterized circuit ($D = 16$, $n = 4$). Empirically, on a binary subset of the 20 Newsgroups corpus, this approach yields improved generalization compared with a shallower real-valued quantum NLP (QNLP) baseline while remaining tractable to simulate on classical resources. This paper should be interpreted as a controlled representational study rather than

© 2026 IOP Publishing Ltd. All rights, including for text and data mining, AI training, and similar technologies, are reserved.

a claim that QCLP outperforms strong classical NLP systems. The binary 20 Newsgroups benchmark is intentionally modest so that we can isolate the effect of complex embeddings, amplitude encoding, and variational measurement from confounding factors such as large-scale pretraining, large model capacity, or long-context reasoning. Accordingly, improvements over the QNLP baseline are evidence that the proposed quantum feature map alters the inductive bias of a compact classifier, not evidence of broad quantum advantage.

A complex-valued classical network could capture part of the phase-related structure explored here, and we do not claim otherwise. The distinction of QCLP is that the embeddings are normalized into quantum states, evolved by unitary operations, and converted into features through measurement. That measurement-constrained pipeline defines a different hypothesis class from an unconstrained classical complex-valued network, and a direct comparison against such a baseline remains an important direction for future work. The following sections describe the complex embedding formalism, our state-preparation strategy, and the four-layer parametric ansatz used for feature extraction.

To the best of our knowledge, there is currently no controlled experimental framework that isolates the role of complex-valued embeddings within a quantum state preparation and measurement pipeline for NLP tasks. This work therefore fills a methodological gap by providing a reproducible setting in which the representational and optimization behavior of such hybrid quantum–classical models can be systematically analyzed.

1.1. Background and motivation

While the bulk of QML research has focused on structured, numerical, or image data, QNLP aims to extend these ideas into the realm of textual information [4]. Early QNLP endeavors have proposed encoding sentences into quantum states via amplitude encoding or angle encoding of real-valued word embeddings, followed by parameterized quantum circuits (PQCs) to extract features for classification, sentiment analysis, or simple grammatical inference tasks [5]. Such approaches are promising, but they remain largely exploratory for several reasons:

1.1.1. Real-valued embedding limitation

Almost all existing QNLP methods rely on embeddings mapped from classical, real-valued vectors. Although amplitude encoding can map a real-valued vector of dimension D into an n-qubit quantum state of dimension 2^n, it does not exploit the fact that quantum state amplitudes are inherently complex. By restricting to real-valued embeddings, prior work may lose expressive power tied to phase information.

1.1.2. Shallow circuits and limited qubit resources

A large share of existing QNLP demonstrations is restricted to extremely small quantum devices, typically using only 2–4 qubits and one or two layers of trainable gates. While such setups are useful for proof-of-concept studies, they also impose a hard cap on the representational capacity of the model: because the embedding dimension must satisfy $D \leqslant 2^n$, only a very small vocabulary or feature space can be mapped directly into a quantum state. In addition, shallow circuits provide limited opportunity for token components to interact through entangling operations, which makes it difficult to model the richer correlations that often appear in natural language, particularly when those correlations depend on subtle phase structure or longer-range dependencies.

1.1.3. Amortized state preparation overhead

Another major bottleneck is the cost of preparing amplitude-encoded states from classical vectors. In the simplest constructions, state preparation grows exponentially with the number of qubits, with gate counts on the order of $O(2^n)$; more advanced preparation schemes can reduce depth, but they usually do so by introducing additional ancilla qubits or more complicated circuit layouts. On near-term quantum hardware, this initialization cost can become a dominant part of the computation and can consume much of the available noise budget, which means that any benefit gained from the quantum feature extraction stage may be offset by the overhead needed to load the data.

To address these limitations, we propose QCLP, which is designed to overcome several weaknesses of earlier QNLP frameworks [6, 7] by:

- learning complex-valued token embeddings so that text representations are formed directly in a complex Hilbert space, matching the mathematical structure of quantum state vectors;
- using a multi-layer entangling ansatz that can model higher-order interactions among amplitude-encoded features more effectively than a single-layer circuit;

- training the complete hybrid system end to end, with the complex embedding parameters, quantum circuit parameters, and classical readout weights optimized jointly through backpropagation together with the parameter-shift rule.

1.2. Overview of contributions

This work delivers a unified treatment of QCLP, spanning foundational theory, system design, and empirical validation. Our main contributions are:

- **Complex embedding formalism.** We define a `ComplexEmbeddingLayer` that encodes each token w as a pair of real vectors $(\mathbf{r}_w, \mathbf{i}_w) \in \mathbb{R}^D \times \mathbb{R}^D$. The token's complex embedding is then given by

$$\mathbf{e}_w = \mathbf{r}_w + i\mathbf{i}_w \in \mathbb{C}^D,$$

  enabling the model to capture both amplitude-like and phase-like components at the token level while maintaining efficient lookup semantics.

- **Amplitude encoding of complex embeddings.** We present a practical aggregation for token-level embeddings into a sentence representation

$$\mathbf{e}_s = \frac{1}{L}\sum_{i=1}^{L} \mathbf{e}_{w_i} \in \mathbb{C}^D,$$

  and describe how to convert $\mathbf{e}_s$ into a normalized quantum state $|\psi_s\rangle \in \mathbb{C}^{2^n}$ via amplitude encoding. We choose

$$n = \lceil \log_2 D \rceil$$

  to guarantee $2^n \geqslant D$. (For our experiments we use $n = 4$ and $D = 16$, allowing direct amplitude encoding into a 4-qubit register.)

- **Four-layer parameterized QCLP circuit.** We design a layered quantum ansatz consisting of four repeated blocks; each block applies local single-qubit rotations (e.g. R_Z, R_Y, R_X) followed by a full entangling chain of controlled-NOT (CNOT) gates. This multi-block structure increases expressiveness compared to a single-layer circuit while remaining mindful of NISQ-era trainability.

- **Hybrid quantum–classical readout and optimization.** The measured Pauli-Z expectation values from the quantum circuit are fed into a compact classical feedforward network, which produces the final binary prediction probability. Training is performed end to end: the classical layers are optimized with backpropagation using AdamW, while the quantum circuit parameters are updated from parameter-shift gradient estimates.

- **Comprehensive empirical evaluation.** We test QCLP on a binary subset of the 20 Newsgroups corpus and compare it with a single-layer, real-valued QNLP baseline as well as a small classical neural model. The reported evaluation includes accuracy, $F1$ score, receiver operating characteristic (ROC) AUC, precision-recall (PR) AUC, log loss, and Matthews correlation coefficient (MCC), with results averaged over multiple random seeds to capture run-to-run variation.

- **Limitations and avenues for future work.** We discuss practical constraints such as state-preparation cost, limited qubit availability, and the effects of normalization coupling, and we identify several extensions for future study, including complex-valued attention, more structured state-preparation methods, and hardware-aware ansätze for larger quantum systems.

1.3. Organization of the paper

The remainder of the paper is organized as follows: section 2 reviews the background and related work, including classical embedding methods, amplitude encoding, and prior QNLP studies. Section 3 presents the methodology, covering the ComplexEmbeddingLayer, the quantum ansatz, and the classical readout model. Section 4 describes the experimental setup, dataset, preprocessing, hyperparameters, baselines, and reproducibility details. Section 5 reports the results, analyses, and visualizations. Section 6 discusses the findings and their implications, and section 7 concludes the paper with key takeaways and future directions.

2. Related work

2.1. Classical NLP and embeddings

Classical NLP methods typically rely on real-valued vector embeddings to represent discrete tokens (words, subwords, characters) in a continuous latent space. Early approaches such as Word2Vec [8] and GloVe [9] learn static embeddings by optimizing co-occurrence statistics. Contextual embeddings such as ELMo and BERT compute token representations with bidirectional context; autoregressive models such as GPT compute token representations conditioned on left-to-right context [10–12]. A common pattern in text classification is to map each token to a D-dimensional real vector, obtain a sentence embedding via mean or max pooling, and feed it into a feedforward neural network for classification [13].

Formally, let $\mathcal{V}$ be the vocabulary of size $|\mathcal{V}| = V$. A learnable embedding matrix $\mathbf{E} \in \mathbb{R}^{V\times D}$ maps token w to $\mathbf{e}_w \in \mathbb{R}^D$. Given a sentence $s = [w_1, w_2, \ldots, w_L]$, one may compute

$$\mathbf{e}_{s_n}^{(\text{mean})} = \frac{1}{L}\sum_{i=1}^{L} \mathbf{e}_{w_i},$$

and then pass $\mathbf{e}_{s_n}^{(\text{mean})}$ through an MLP with parameters $\mathbf{W}^{(1)}, \mathbf{b}^{(1)}, \mathbf{W}^{(2)}, \mathbf{b}^{(2)}$ to obtain logits. Training minimizes binary cross-entropy:

$$\mathcal{L} = -\frac{1}{N}\sum_{n=1}^{N} [y_n \log \hat{y}_n + (1-y_n)\log(1-\hat{y}_n)],$$

where $\hat{y}_n = \sigma\big(\mathbf{W}^{(2)} \cdot \text{ReLU}(\mathbf{W}^{(1)}\mathbf{e}_{s_n} + \mathbf{b}^{(1)}) + \mathbf{b}^{(2)}\big)$.

Despite their success, classical embeddings are strictly real-valued. By moving to quantum representations, one can potentially leverage superposition and entanglement to encode richer relationships.

2.2. Quantum computing and quantum machine learning

A quantum bit (qubit) [4, 14] resides in a two-dimensional Hilbert space spanned by $\{|0\rangle, |1\rangle\}$, where a general pure state is

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \quad \alpha, \beta \in \mathbb{C}, \quad |\alpha|^2 + |\beta|^2 = 1.$$

An n-qubit register hence lives in a 2^n-dimensional complex Hilbert space [15] with computational basis states $\{|b_{n-1}b_{n-2}\cdots b_0\rangle : b_i \in \{0,1\}\}$. A statevector $|\Psi\rangle = \sum_{j=0}^{2^n-1} c_j|j\rangle$ has complex amplitudes $c_j \in \mathbb{C}$ satisfying $\sum_j |c_j|^2 = 1$. Unitary transformations U act linearly on the state: $|\Psi'\rangle = U|\Psi\rangle$.

PQCs [16, 17] have emerged as quantum analogues of neural networks [7]. A PQC applies a sequence of parameterized gates such as single-qubit rotations $R_\alpha(\theta) = \mathrm{e}^{-\mathrm{i}\theta\sigma_\alpha/2}$ (for $\alpha \in \{X, Y, Z\}$) and CNOT entangling gates to an initial state $|0\rangle^{\otimes n}$ or to a state prepared via amplitude encoding [18, 19]. The resulting state $|\Psi(\boldsymbol{\theta})\rangle$ depends continuously on parameters $\boldsymbol{\theta} \in \mathbb{R}^m$. By measuring expectation values of observables (Pauli-Z on each qubit), one obtains features $\boldsymbol{f}(\boldsymbol{\theta}) \in \mathbb{R}^n$ that can be fed into a classical loss function [20]. Training proceeds by updating $\boldsymbol{\theta}$ to minimize a cost, commonly via the parameter-shift rule:

$$\frac{\partial\langle O\rangle}{\partial\theta_j} = \frac{1}{2}\left[\langle O\rangle\left(\theta_j + \frac{\pi}{2}\right) - \langle O\rangle\left(\theta_j - \frac{\pi}{2}\right)\right].$$

Hybrid quantum–classical loops [21, 22] have been used in applications such as classification [23], generative models, and reinforcement learning [24]. However, most QML works focus on real-valued data [3, 25] or amplitude-encoded classical features [6]. Our QCLP, in contrast, naturally leverages complex statevectors to represent complex embeddings [26].

2.3. QNLP and previous approaches

Prior work on QNLP as demonstrated in table 1 [27–29], typically employs real embeddings and focuses on grammar-inspired models [30–32] such as DisCoCat (distributional compositional categorical) [33–35] frameworks. In such approaches, one encodes word vectors [36, 37] into small numbers of qubits (often with $n = 2$ or 3) via amplitude encoding and applies shallow circuits parameterized single-layer entanglers) to extract features [38–40]. The readout often uses expectation values or projective measurements to classify sentiment or topic [41, 42]. While promising, these models have two main limitations:

Table 1. Literature review of prior approaches.

Author & Year	Major findings	Technique	Model / Framework	Dataset / Task	Key results / impact	Scope / Limitations
Coecke, Sadrzadeh & Clark (2010)	Proposed a categorical compositional model uniting grammar and distributional semantics.	Category theory, tensor-based semantics	DisCoCat framework	Theoretical linguistic examples	Established the mathematical basis for compositional semantics in quantum formalisms	Theoretical only; lacks empirical validation
Schuld & Killoran (2019)	Defined quantum feature spaces enabling kernel-based learning in Hilbert space.	Quantum kernel methods, feature Hilbert spaces	Quantum feature maps for ML	Benchmark ML tasks (small datasets)	Demonstrated link between classical kernels and quantum state encodings	General QML method, not NLP-specific
Lorenz *et al* (2021)	Conducted first NLP experiments on real quantum hardware.	Compositional-to-circuit mapping	QNLP pipeline (DisCoCat → circuit)	Sentence classification (100+ sentences)	Demonstrated convergence on IBM Q hardware and simulators	Hardware noise and dataset size limitations
Meichanetzidis *et al* (2020)	Introduced grammar-aware sentence classification using quantum circuits.	Syntax-aware quantum circuit simulation	QNLP classifier with grammatical structure	Simulated sentence classification tasks	Improved accuracy over syntax-agnostic circuits	Simulated only; small-scale experiments
Coecke *et al* (2020)	Proposed theoretical and architectural foundations for near-term QNLP.	Diagrammatic-to-circuit translation, DisCoCat mapping	Near-term QNLP framework for NISQ devices	Conceptual and simulated examples	Established design principles for implementable QNLP	Theoretical, limited empirical evaluation
O'Riordan *et al* (2020)	Developed a hybrid classical-quantum workflow for NLP.	Hybrid encoding + decoding pipeline	Classical-quantum hybrid model	Small NLP tasks	Demonstrated feasibility of hybrid workflows	Scaling and generalization challenges remain
Our Work (2025)	Integrated complex embeddings, amplitude encoding, and PQC-based readout for semantic representation.	Complex-valued embedding → amplitude encoding → PQC	QCLP Framework (Quantum Complex Language Processing)	Binary text classification (subset of 20 Newsgroups)	Outperforms QNLP baselines	Simulation-based; scalability to larger qubit counts remains future work

- **Real-valued embeddings:** They do not explicitly exploit the inherently complex Hilbert space of quantum mechanics [29].
- **Shallow circuits:** Many QNLP experiments use one or two circuit layers, limiting expressive capacity.

Our QCLP augments these by introducing complex embeddings that directly match the quantum state's domain and employing a deeper (four-layer) entangling circuit to extract more sophisticated correlations.

2.4. Complex Hilbert spaces and complex embeddings

Complex embeddings [43, 44] have been explored in classical settings (Complex-valued word embeddings) to model asymmetric relationships [45] (like word analogies) or to capture phase information [46, 47]. A complex embedding for word w is

$$\mathbf{e}_w = \mathbf{r}_w + i\mathbf{i}_w, \quad \mathbf{r}_w, \mathbf{i}_w \in \mathbb{R}^D.$$

Operations such as cosine similarity extend naturally:

$$\mathrm{sim}(w_1, w_2) = \frac{\mathrm{Re}\left(\langle \mathbf{e}_{w_1}, \mathbf{e}_{w_2} \rangle\right)}{\|\mathbf{e}_{w_1}\| \|\mathbf{e}_{w_2}\|}, \quad \langle \mathbf{e}_{w_1}, \mathbf{e}_{w_2} \rangle = \sum_{j=0}^{D-1} \mathbf{e}^*_{w_1,j} \mathbf{e}_{w_2,j}.$$

In quantum mechanics, any state is a normalized complex vector in $\mathbb{C}^{2^n}$. By aligning complex embeddings with amplitude encodings, one can directly prepare the embedding as a quantum state without discarding phase information [3]. This motivates our approach: maintain embeddings in $\mathbb{C}^D$, aggregate to a sentence embedding in $\mathbb{C}^D$, and then amplitude-encode into $\mathbb{C}^{2^n}$.

Recent QNLP work has begun to explore quantum-native attention mechanisms and hybrid transformer-like architectures that integrate quantum kernels with self-attention (see recent works on quantum self-attention and quantum-enhanced attention mechanisms). Although the QCLP framework presented here does not implement a full quantum self-attention module, the extension to complex-valued attention mechanisms [48] is straightforward (section VI) and forms clear future work to bridge QCLP with emerging quantum transformer architectures.

In the proposed QCLP architecture, cosine similarity between embeddings is not directly computed within the quantum circuit. Instead, sentence embeddings are first amplitude-encoded into a normalized quantum state, after which a parameterized quantum circuit transforms the state through a sequence of rotation and entangling operations. The circuit outputs a vector of Pauli-Z expectation values measured on each qubit. These expectation values form a feature representation that is subsequently processed by a classical readout network.

To avoid ambiguity, the real-part cosine similarity expression is presented as a conceptual comparison with classical embedding methods rather than as an operation computed inside the quantum circuit. In the implementation, QCLP prepares a normalized quantum state from the complex embedding and then extracts features through expectation values of measured observables, specifically Pauli-Z operators after the variational circuit. The classification signal therefore comes from measured quantum statistics, not from an explicit cosine-similarity computation. In this sense, the model is quantum because its representations are constrained by unitary evolution and measurement, even though the cosine-similarity discussion is used only to motivate the embedding geometry.

Therefore, the cosine similarity formulation discussed earlier should be interpreted only as an intuitive analogy to classical embedding methods rather than an operation explicitly implemented in the quantum circuit. Unlike fidelity-based quantum classifiers, which directly measure overlaps between quantum states, QCLP relies on expectation-value features extracted from the variational circuit.

3. Methodology

We now present an in-depth description of each component of the QCLP model. We discuss (i) how complex representations are learned via a dual-matrix embedding layer, (ii) how these embeddings are aggregated and converted into a normalized quantum state, (iii) the structure and rationale of our four-layer parameterized quantum circuit, (iv) the design of the classical feedforward readout, and (v) the overall objective and optimization strategy. We have also formulated a simple workflow diagram as illustrated in figure 1, for better understanding of our procedure.

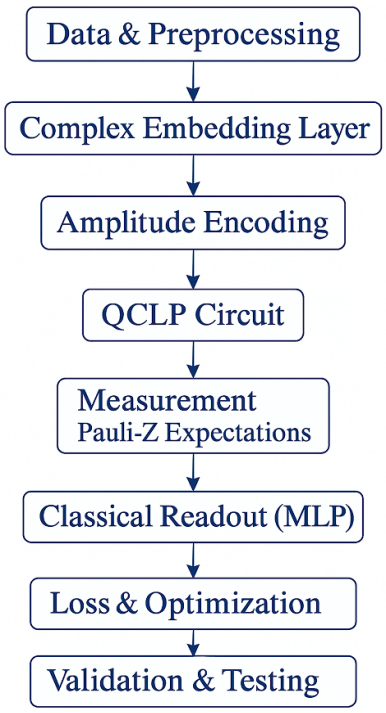


Figure 1. Process Workflow Diagram.

3.1. Complex embedding layer

At the foundation of QCLP is a `ComplexEmbeddingLayer`, which extends classical real-valued embeddings into the complex plane. Rather than mapping each word to a single real vector, we learn two parallel embedding matrices so that each token's representation is naturally a vector in $\mathbb{C}^D$.

3.1.1. Theoretical motivation for complex embeddings

Real-valued embeddings capture amplitude-like similarities via dot products. Complex-valued embeddings extend representational capacity by introducing a phase degree of freedom per token component: for tokens w_1, w_2 with complex embeddings $e_{w_1}, e_{w_2} \in \mathbb{C}^D$, the inner product is

$$\langle e_{w_1}, e_{w_2} \rangle = \sum_j \overline{e_{w_1,j}}\, e_{w_2,j}.$$

The real part $\mathrm{Re}(\langle e_{w_1}, e_{w_2} \rangle)$ captures amplitude-aligned similarity while the imaginary part captures relative phase/asymmetry. Phase allows the representation of inherently asymmetric relations (for example, directional relations in linguistics such as hypernymy or subject-object roles), because the complex inner product is not constrained to be symmetric in its real-imaginary decomposition.

Concretely, consider two tokens whose semantics differ primarily by relational context (e.g. buy vs sell). By encoding a phase shift in components corresponding to financial context, the complex inner product can make buy and sell distinct under interferometric measurements performed by PQCs while preserving amplitude similarities in topical components. From a quantum perspective, complex amplitudes permit interference phenomena: when two token vectors combine within the amplitude-encoded state, their relative phases cause constructive or destructive interference across basis states, enabling richer compositional effects than purely real embeddings. We therefore view complex embeddings as a low-cost method to introduce an additional representational axis (phase) that PQCs can exploit to form interference-based features.

While the above is not a formal proof of universal superiority, it provides a principled explanation for the empirical gains observed when moving from real-valued amplitude encodings to complex-valued embeddings in our experiments; we complement this discussion with empirical ablations (section 5.2) which quantify the incremental benefit.

3.1.2. Architecture and parameterization

3.1.2.1. Embedding matrices

Let $\mathcal{V}$ denote the vocabulary of size $|\mathcal{V}| = V$. We introduce two matrices:

$$R = \left[r_{w,j}\right]_{w\in\mathcal{V},\,1\leqslant j\leqslant D} \quad \text{and} \quad I = \left[i_{w,j}\right]_{w\in\mathcal{V},\,1\leqslant j\leqslant D},$$

each of shape $V \times D$, where D is the embedding dimension. For each token $w \in \mathcal{V}$, we define

$$r_w = (r_{w,1},\ \ldots,\ r_{w,D})^\top \in \mathbb{R}^D, \quad i_w = (i_{w,1},\ \ldots,\ i_{w,D})^\top \in \mathbb{R}^D.$$

The corresponding complex embedding is

$$e_w = r_w + i\, i_w \in \mathbb{C}^D.$$

Thus the total number of learnable parameters in the embedding layer is $2V \times D$.

3.1.2.2. Token-to-embedding lookup

During the forward pass, an input mini-batch of size B is represented as an integer tensor $X \in \mathbb{Z}_+^{B\times L}$, where L is the maximum sentence length (padding shorter sentences with a special <pad> token of index 0). The embedding layer produces, for each token index w, the complex vector e_w. Therefore, the output of this layer is a tensor

$$E = \left[e_{w_{b,1}}, e_{w_{b,2}}, \ldots, e_{w_{b,L}}\right]_{b=1,\ldots,B} \in \mathbb{C}^{B\times L\times D}.$$

3.1.3. Initialization and regularization

3.1.3.1. Weight initialization

Both R and I are initialized with small random values, typically drawn from a normal distribution $\mathcal{N}(0,\sigma^2)$ with $\sigma = 0.01$. This ensures that initially, the complex embeddings have modest magnitudes and uniformly distributed phases.

3.1.3.2. Embedding dimension D

The choice of D is constrained by the number of qubits n available for amplitude encoding, since we require $D \leqslant 2^n$. In our primary experiments, we fix $n = 4$ and therefore set $D = 16$. In more general settings, D can be chosen up to 2^n. A larger D allows richer token representations but increases the state-preparation cost and the number of quantum parameters in downstream layers.

3.1.3.3. Regularization

We apply ℓ_2 weight decay to both embedding matrices R and I with a small coefficient 10^{-4}). This discourages runaway norms in the embeddings and stabilizes training. No further dropout or batch-normalization is used inside the embedding layer, as tokens are discrete and the primary regularization occurs at the readout stage.

3.2. Sentence embedding and quantum state preparation

After obtaining token-level embeddings $E \in \mathbb{C}^{B\times L\times D}$, we must compute, for each example, a single complex vector of dimension D and then map it to a normalized n-qubit state in $\mathbb{C}^{2^n}$.

3.2.1. Mean-pooling to obtain sentence embeddings

3.2.1.1. Mean-pooling operation

For each sentence $s^{(b)}$ in the batch, with L tokens $(w_{b,1}, w_{b,2}, \ldots, w_{b,L})$, we define the mean-pooled complex vector

$$e_s^{(b)} = \frac{1}{L}\sum_{j=1}^{L} e_{w_{b,j}} = u_s^{(b)} + i\, v_s^{(b)} \in \mathbb{C}^D,$$

where

$$u_s^{(b)} = \frac{1}{L}\sum_{j=1}^{L} r_{w_{b,j}}, \quad v_s^{(b)} = \frac{1}{L}\sum_{j=1}^{L} i_{w_{b,j}}.$$

Denote by $E_{\text{mean}} \in \mathbb{C}^{B\times D}$ the tensor whose bth row is $e_s^{(b)}$. [49]

3.2.1.2. Rationale for mean-pooling
Although more sophisticated pooling strategies such as attention-weighted sums could be employed, mean-pooling provides a simple, permutation-invariant way to aggregate token embeddings. In the context of QCLP, it also simplifies gradient flow and ensures a fixed-size representation regardless of sentence length. Future work might explore complex-valued attention mechanisms, but we defer that to follow-up studies.

3.2.2. Amplitude encoding into an n-qubit state
Let each sentence embedding for example b be

$$e_s(b) = (e_{s,0}, e_{s,1}, \ldots, e_{s,D-1})^\top \in \mathbb{C}^D.$$

Let $N = 2^n$ and suppose $N \geqslant D$. We form an N-dimensional complex vector $\tilde{z}_s(b) \in \mathbb{C}^N$ by zero-padding:

$$\tilde{z}_s(b) = (e_{s,0}, e_{s,1}, \ldots, e_{s,D-1}, 0, \ldots, 0)^\top.$$

Define the squared ℓ_2-norm

$$Z_s(b) = \sum_{j=0}^{D-1} |e_{s,j}|^2 = \|\tilde{z}_s(b)\|_2^2 \geqslant 0.$$

The normalized amplitude vector $z_s(b) \in \mathbb{C}^N$ is then obtained by

$$z_s(b) = \frac{\tilde{z}_s(b)}{\sqrt{Z_s(b) + \varepsilon}},$$

with a small stability constant $\varepsilon = 10^{-8}$. This choice correctly normalizes

$$\|z_s(b)\|_2 \approx 1$$

and ensures differentiability when $Z_s(b)$ is (near) zero.

The n-qubit state is

$$|\psi_s(b)\rangle = \sum_{j=0}^{N-1} z_{s,j}(b)\, |j\rangle, \quad \|z_s(b)\|_2 = 1.$$

3.2.3. Complexity of amplitude encoding
3.2.3.1. Gate count and depth
Preparing an arbitrary N-dimensional quantum state via general decomposition typically requires a gate count that scales on the order of $O(N)$ elementary one- and two-qubit gates in exact decompositions; structured or approximate encodings (tree-based decompositions, block-encodings, or QRAM-assisted approaches) can reduce depth at the cost of extra ancilla qubits or approximation error. We replace the previous reference with more targeted sources on state-preparation complexity (see e.g. Mottonen *et al* 'Transformation of quantum states using uniformly controlled rotations' and the review by Shende *et al*). [50, 51]

3.2.3.2. Gradient flow through normalization
Differentiation through the normalization couples components of the pre-normalized vector. Let $\tilde{z}_s$ denote the zero-padded (unnormalized) vector and $Z_s = \|\tilde{z}_s\|_2^2$. For $0 \leqslant p, j < D$,

$$\frac{\partial z_{s,p}}{\partial e_{s,j}} = \frac{\delta_{p,j}}{\sqrt{Z_s + \varepsilon}} - \frac{\tilde{z}_{s,p}\, \overline{\tilde{z}_{s,j}}}{(Z_s + \varepsilon)^{3/2}},$$

where $(\cdot)^-$ denotes complex conjugation and $\delta_{p,j}$ is the Kronecker delta. Automatic-differentiation frameworks that treat real and imaginary parts explicitly (or support complex differentiation) correctly propagate gradients using the above expression.

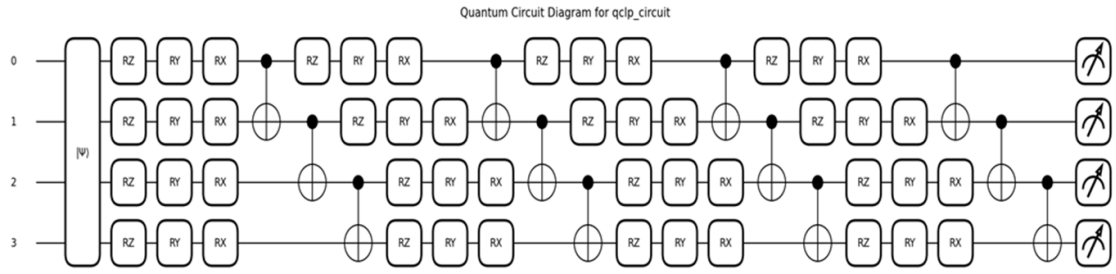


Figure 2. QCLP Circuit Diagram.

3.3. QCLP circuit design

Once each sentence is encoded as an n-qubit state $|\psi_s\rangle$, the QCLP model applies a parameterized quantum circuit $U(\theta)$ as shown in figure 2, that is composed of four successive layers, each featuring local rotations and a linear chain of CNOT entanglers. This section details the ansatz, its expressive power, and implementation considerations.

3.3.1. Overall circuit structure

For n qubits ($n=4$ in our experiments, but extensible to any $n \leqslant$ hardware capacity), label the qubits $Q_0, Q_1, \ldots, Q_{n-1}$. We define $U(\theta)$ as

$$U(\theta) = U_{\text{layer},3} \times U_{\text{layer},2} \times U_{\text{layer},1} \times U_{\text{layer},0},$$

$$U_{\text{layer},l} = \left(\bigotimes_{i=0}^{n-1} R_X(\theta_{l,i,2})\, R_Y(\theta_{l,i,1})\, R_Z(\theta_{l,i,0}) \right) \left(\prod_{i=0}^{n-2} \text{CNOT}_{Q_i \to Q_{i+1}} \right),$$

where:

- $l \in \{0,1,2,3\}$ indexes the four layers;
- $\theta_{l,i,0}, \theta_{l,i,1}, \theta_{l,i,2} \in \mathbb{R}$ are the rotation angles for qubit i in layer l corresponding to R_Z, R_Y, and R_X respectively;

Each single-qubit rotation $R_\alpha(\phi)$ acts on one qubit and has the form:

$$R_\alpha(\phi) = \exp\left(-\mathrm{i}\frac{\phi}{2}\sigma_\alpha\right), \quad \alpha \in \{X,Y,Z\}, \quad \sigma_\alpha = \text{Pauli-}\alpha \in \mathbb{C}^{2\times 2}.$$

The entangling block is a chain of $(n-1)$ CNOT gates: $\text{CNOT}_{Q_0 \to Q_1}$, $\text{CNOT}_{Q_1 \to Q_2}$, …, $\text{CNOT}_{Q_{n-2} \to Q_{n-1}}$. Each CNOT introduces two-qubit entanglement between adjacent qubits in a linear topology.

In total, the parameter tensor θ has shape $(4, n, 3)$, containing $12n$ real parameters. For $n=4$, this is 48 parameters.

3.3.2. Implementation and initialization

We provide here concrete implementation details and initialization strategies that influence trainability and reproducibility. The PQC is implemented with PennyLane (or Qiskit / other framework) using state-vector simulation for all experiments reported in this manuscript. Each rotation parameter $\theta_{l,i,j}$ is instantiated as a PyTorch `nn.Parameter` (or the framework equivalent) and optimized jointly with classical parameters via AdamW.

3.3.2.1. Parameter initialization

To reduce early-stage vanishing gradients and to bias the circuit to the identity, rotation parameters are sampled from

$$\theta_{l,i,j} \sim \mathcal{N}\left(0, \sigma_0^2\right), \quad \sigma_0 = 0.02.$$

This near-identity initialization keeps $U(\theta) \approx I$ initially, preserving the information encoded in $|\psi_s\rangle$ and empirically helps maintain meaningful gradient magnitudes in early epochs.

3.3.2.2. Gradient clipping
We clip the ℓ_2-norm of quantum-parameter gradients per update to a hard threshold

$$G_{\max} = 2.0.$$

In practice this stabilizes updates and prevents rare large steps that can destroy the encoded amplitudes.

3.3.3. Single-qubit rotations
Each layer's single-qubit rotations allow arbitrary rotations on the Bloch sphere for each qubit, up to a global phase. By concatenating R_Z, R_Y, and R_X in that order, we obtain a universal single-qubit unitary (up to an irrelevant global phase):

$$R_X(\alpha)R_Y(\beta)R_Z(\gamma) \propto \begin{bmatrix} \cos\left(\frac{\alpha}{2}\right)\cos\left(\frac{\beta}{2}\right)e^{-i\frac{\gamma}{2}} - \sin\left(\frac{\alpha}{2}\right)\sin\left(\frac{\beta}{2}\right)e^{i\frac{\gamma}{2}} & \cos\left(\frac{\alpha}{2}\right)\sin\left(\frac{\beta}{2}\right)e^{i\frac{\gamma}{2}} - \sin\left(\frac{\alpha}{2}\right)\cos\left(\frac{\beta}{2}\right)e^{-i\frac{\gamma}{2}} \\ -\cos\left(\frac{\alpha}{2}\right)\sin\left(\frac{\beta}{2}\right)e^{-i\frac{\gamma}{2}} - \sin\left(\frac{\alpha}{2}\right)\cos\left(\frac{\beta}{2}\right)e^{i\frac{\gamma}{2}} & \cos\left(\frac{\alpha}{2}\right)\cos\left(\frac{\beta}{2}\right)e^{i\frac{\gamma}{2}} + \sin\left(\frac{\alpha}{2}\right)\sin\left(\frac{\beta}{2}\right)e^{-i\frac{\gamma}{2}} \end{bmatrix} \tag{1}$$

which spans the full SU(2) up to a global phase. By choosing three real parameters per qubit, we allow maximal expressivity for local rotations.

3.3.4. Full entangling chain
The entangler $\prod_{i=0}^{n-2}\mathrm{CNOT}_{Q_i \to Q_{i+1}}$ creates a linear chain of entanglement: qubit Q_0 is entangled with Q_1, then qubit Q_1 with Q_2, and so on. Over four layers, this ensures that information can propagate across all qubits, albeit sequentially. Such a linear entangler is:

- **Hardware-efficient:** On many platforms where nearest-neighbor connectivity is a natural layout superconducting qubit processors laid out in a line).
- **Expressive enough:** To generate multipartite entanglement: after two layers, e.g. Q_0 and Q_2 can become indirectly entangled via Q_1.
- **Simple to simulate**: In classical simulation, CNOT gates require only $\mathcal{O}(1)$ updates in statevector; on hardware, the depth is minimal if qubits are arranged linearly.

Alternative entangling patterns all-to-all pairs, brickwall architectures, or ring topologies) may improve expressivity but can increase circuit depth or conflict with hardware layouts.

3.3.5. Expressive power and barren plateau considerations
3.3.5.1. Expressivity
With 4 layers, each adding local rotations and entanglers, the circuit can, in principle, approximate any unitary in a certain subspace of $\mathrm{SU}(2^n)$ up to gauge. While four layers on $n=4$ qubits do not form a universal unitary 2-design, they constitute a hardware-efficient ansatz capable of capturing complex correlations among the 16 amplitudes $\{z_j\}$. In particular, each layer's three parameters per qubit allow local reorientation of basis states, while CNOTs propagate amplitude and phase information across qubits.

3.3.5.2. Parameter count vs. expressivity trade-off
Increasing the number of circuit layers ($L>4$) or augmenting the entangling resources expands the set of reachable states, but introduces two practical costs: (i) deeper circuits that incur larger noise on NISQ devices, and (ii) a greater risk of encountering barren-plateau phenomena, i.e. optimization landscapes with exponentially vanishing gradients [52]. In our empirical study, a four-layer ansatz with $n=4$ offers a pragmatic compromise between representational power and trainability: at random initialization the gradient norms are typically on the order of 10^{-2}–10^{-1} and we do not observe pronounced plateauing.

3.3.6. Forward pass and output features
Given a sentence s, after preparing the amplitude-encoded state $|\psi_s\rangle$ we apply the parameterized unitary $U(\theta)$ to obtain the circuit output state

$$|\phi_s(\theta)\rangle = U(\theta)\,|\psi_s\rangle\,.$$

We then evaluate the Pauli-Z expectation on each qubit. For qubit Q_i (with $i=0,\ldots,n-1$) the measured feature is

$$f_i(s;\theta) = \langle\phi_s(\theta)|\,Z_i\,|\phi_s(\theta)\rangle \in [-1,1]\,,$$

which is a smooth function of the variational parameters θ and of the encoded amplitudes z_s. Stacking the per-qubit expectations yields the quantum feature vector

$$f(s;\theta) = [f_0(s;\theta), f_1(s;\theta), \ldots, f_{n-1}(s;\theta)]^\top \in \mathbb{R}^n.$$

For a mini-batch of size B these features assemble into a tensor $F \in \mathbb{R}^{B\times n}$ that is consumed by the classical readout.

3.4. Classical readout network

The classification head is a deliberately compact classical feedforward module that maps the n-dimensional quantum feature vector $\mathbf{f}$ to a scalar probability $\hat{y} \in (0,1)$. Keeping this module small helps isolate the contribution of the quantum feature extractor.

3.4.1. Network architecture

Let $\mathbf{f} \in \mathbb{R}^n$ denote the quantum-derived feature vector for a single sentence. We use a two-layer perceptron with hidden dimension m:

- **First linear layer:** parameters $W^{(1)} \in \mathbb{R}^{m\times n}$ and $b^{(1)} \in \mathbb{R}^m$, producing the hidden activations

$$h = \sigma\left(W^{(1)}\mathbf{f} + b^{(1)}\right) \in \mathbb{R}^m,$$

  where $\sigma(\cdot)$ denotes a nonlinearity (e.g. ReLU).

- **Second linear layer:** parameters $W^{(2)} \in \mathbb{R}^{1\times m}$ and $b^{(2)} \in \mathbb{R}$, producing the logit

$$\ell = W^{(2)}h + b^{(2)} \in \mathbb{R},$$

  and the predicted probability is obtained via the sigmoid:

$$\hat{y} = \text{sigmoid}(\ell) \in (0,1).$$

The computation proceeds as:

$$\mathbf{z} = W^{(1)}\mathbf{f} + b^{(1)}, \quad \mathbf{z} \in \mathbb{R}^m.$$
$$\mathbf{h} = \text{ReLU}(\mathbf{z}), \quad h_j = \max(0, z_j), \quad j = 1, \ldots, m.$$

Dropout: randomly zero-out 10% of the entries of $\mathbf{h}$ during training, obtaining $\mathbf{h}'$. At inference, scale activations by a factor $1 - 0.1 = 0.9$ or equivalently disable dropout.

Compute scalar logit:

$$\ell = W^{(2)}\mathbf{h}' + b^{(2)}, \quad \ell \in \mathbb{R}.$$

Apply sigmoid:

$$\hat{y} = \sigma(\ell) = \frac{1}{1+e^{-\ell}}.$$

Hence, the network has

$$\underbrace{mn + m}_{\text{first layer}} + \underbrace{m + 1}_{\text{second layer}} = m(n+2) + 1$$

real parameters. For $n = 4$, $m = 8$, that is $8 \times 6 + 1 = 49$ parameters.

3.4.2. Motivation for architecture choices

Hidden dimension $m = 8$:

We choose $m = 8$ to provide sufficient capacity to combine the four quantum expectation values $f_0, \ldots, f_3$ into a more expressive representation, while keeping the overall parameter count small. In preliminary experiments, $m = 8$ produced consistently strong performance; larger m yielded marginal gains at the cost of increased overfitting risk.

ReLU activation:

The rectified linear unit $\mathrm{ReLU}(x) = \max(0, x)$ is standard in modern deep learning due to its simplicity, non-saturating gradient for $x > 0$, and computational efficiency. It introduces nonlinearity, enabling the network to learn threshold-like combinations of quantum features.

Dropout regularization:

We apply dropout with probability $p = 0.1$ to the hidden activations $\mathbf{h}$. Dropout helps prevent co-adaptation of neurons and reduces overfitting, especially when training on moderate-sized datasets (the 20 Newsgroups subset of a few thousand examples). Empirically, $p = 0.1$ strikes a good balance between regularization strength and training stability.

Sigmoid output:

Since our task is binary classification, a single sigmoid output $\hat{y} \in (0, 1)$ suffices. More complex tasks multi-class classification) would require a softmax layer over multiple logits.

3.4.3. Integration with quantum features

During the forward pass, for each minibatch of size B, once we obtain the quantum feature matrix $F \in \mathbb{R}^{B \times n}$, we apply the classical readout to each row independently, producing a prediction vector $\hat{y} \in (0, 1)^B$. Gradients computed w.r.t. F are then propagated back through the quantum module as described in section 3.5.

3.5. Loss function and optimization

To train QCLP in an end-to-end fashion, we minimize the average binary cross-entropy over a mini-batch, while applying appropriate regularization and scheduling strategies.

3.5.1. Binary cross-entropy loss

Given B sentence-label pairs $\{(s^{(b)}, y^{(b)})\}_{b=1}^{B}$, with $y^{(b)} \in \{0, 1\}$ and predicted probability $\hat{y}^{(b)} = C(f(s^{(b)}; \theta))$, the per-example loss is

$$\ell^{(b)} = -\left[y^{(b)} \log \hat{y}^{(b)} + \left(1 - y^{(b)}\right) \log\left(1 - \hat{y}^{(b)}\right)\right].$$

The batch-averaged loss is

$$L = -\frac{1}{B} \sum_{b=1}^{B} \left[y^{(b)} \log \hat{y}^{(b)} + \left(1 - y^{(b)}\right) \log\left(1 - \hat{y}^{(b)}\right)\right].$$

We optionally add an ℓ_2 regularization term on classical weights and embeddings (but not on quantum parameters) of the form

$$L_{\mathrm{reg}} = \lambda_{\mathrm{emb}} \|R\|_F{}^2 + \lambda_{\mathrm{emb}} \|I\|_F{}^2 + \lambda_{\mathrm{cls}} \left(\|W^{(1)}\|_F{}^2 + \|W^{(2)}\|_F{}^2\right),$$

with regularization coefficients $\lambda_{\mathrm{emb}} = \lambda_{\mathrm{cls}} = 10^{-4}$. The final objective is $L_{\mathrm{total}} = L + L_{\mathrm{reg}}$.

3.5.2. Optimization strategy

3.5.2.1. Adamw optimizer

For all classical parameters (R, I, $W^{(1)}$, $b^{(1)}$, $W^{(2)}$, $b^{(2)}$), we apply the AdamW update rule with learning rate $\eta = 0.01$ and weight decay $\texttt{weight_decay} = 10^{-4}$. AdamW decouples ℓ_2 regularization from the gradient step, improving generalization over naive weight decay.

3.5.2.2. Quantum parameter updates

The quantum circuit parameters θ are also updated via AdamW with the same base learning rate $\eta = 0.01$ but with no weight decay. Quantum parameters represent rotation angles on $\mathrm{SU}(2^n)$, where ℓ_2 decay would introduce an unintended bias toward zero rotations, it is preferable to regularize only classical modules.

To prevent issues like barren plateaus or unstable updates, we use gradient clipping on the quantum gradients.

$$\nabla_\theta \leftarrow \frac{\nabla_\theta}{\max\left(1, \frac{\|\nabla_\theta\|_2}{G_{\max}}\right)}, \quad G_{\max} = 2.0.$$

This ensures that the Euclidean norm of the quantum-parameter gradient never exceeds 2.0, which prevents sudden jumps in parameter space that might lead to performance collapse.

3.5.2.3. Learning rate scheduler
We employ a Cosine Annealing schedule with $T_{\max} = 10$ epochs. Concretely, after each epoch t, the learning rate is updated as

$$\eta(t) = \eta_{\min} + \frac{1}{2}(\eta_0 - \eta_{\min})\left(1 + \cos\left(\frac{\pi t}{T_{\max}}\right)\right),$$

where $\eta_0 = 0.01$ is the initial learning rate and $\eta_{\min} = 10^{-4}$. Cosine-annealing helps the model converge smoothly by gradually reducing the step-size as training progresses.

3.5.2.4. Batch size and number of epochs
In our experiments, we set the batch size $B = 64$ and train for 10 epochs. Empirically, this provides sufficient convergence for the 20 Newsgroups binary classification task. Larger datasets or deeper circuits may require more epochs or dynamic batch sizes.

3.5.3. End-to-end backpropagation
3.5.3.1. Classical readout gradients
Standard automatic differentiation computes $\frac{\partial L}{\partial W^{(2)}}, \ldots, \frac{\partial L}{\partial b^{(1)}}$, and produces the upstream gradient $\nabla_f L \in \mathbb{R}^n$ to the quantum module.

3.5.3.2. Quantum gradients via parameter-shift
For each $\theta_{l,i,j}$, we estimate $\frac{\partial L}{\partial \theta_{l,i,j}}$ by applying the parameter-shift rule to compute $\frac{\partial f_k}{\partial \theta_{l,i,j}}$ for each measurement index k, then combining with $\nabla_{f_k} L$. This requires two additional evaluations of the circuit per parameter per batch totaling $2 \times (12n)$ circuit runs for $n = 4$ (i.e. 96 extra runs) plus the forward pass. In practice, PennyLane automatically batches these computations to leverage parallel simulator calls.

3.5.3.3. Embedding gradients through normalization
The gradient $\nabla_z L \in \mathbb{C}^N$ obtained from the quantum module is backpropagated through the normalization step to yield $\nabla_{e_s} L \in \mathbb{C}^D$. Thereafter, we distribute $\frac{\partial L}{\partial e_s}$ evenly to each token's real and imaginary embedding vectors (r_{w_i}, i_{w_i}), scaled by $1/L$.

Implementing these gradient flows within a modern differentiable quantum–classical framework smoothly manages complex calculations, normalization, and derivatives of quantum expectations, allowing for real end-to-end training.

4. Experimental setup

To systematically evaluate the performance and characteristics of QCLP, we conduct experiments on a controlled binary text classification task. In this section, we describe (1) the dataset and preprocessing pipeline; (2) the baseline models used for comparison; (3) implementation specifics including hardware, hyperparameter settings, and training protocols; and (4) the evaluation metrics used to assess classification performance.

4.1. Dataset: 20 newsgroups binary classification

4.1.1. Choice of dataset
We use a carefully selected binary subset of the 20 Newsgroups corpus [53], restricted to the two categories `comp.graphics` and `rec.motorcycles`. The full corpus contains roughly 20 000 documents distributed across 20 topics, but narrowing the problem to two clearly separated classes gives us a controlled binary classification setting with a relatively balanced label distribution. Because the chosen topics are far apart semantically—one centered on computer graphics and the other on motorcycle discussion—the task contains limited label ambiguity and allows us to focus on the effect of the model architecture rather than on challenging class overlap or severe imbalance.

We deliberately use this binary subset as a controlled benchmark. The task is not designed to be a difficult test of broad language understanding, compositional reasoning, or long-range discourse modeling; instead, it is intended to probe how the proposed quantum representation behaves under a simple and reproducible setting. For that reason, the results should be interpreted as benchmark-specific and exploratory, rather than as evidence of quantum advantage in general NLP.

4.1.2. Data acquisition and statistics
After retaining only the `comp.graphics` and `rec.motorcycles` categories, the resulting dataset contains about 2000 documents in total, with each class contributing close to 1000 samples. The class

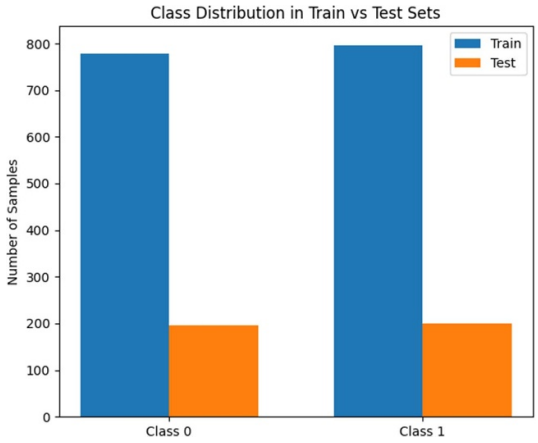


Figure 3. Class Distribution in Train vs Test Sets.

counts are approximately balanced, although small differences may occur (for example, one category may contain slightly more posts than the other).

Train/Test Split. As illustrated in figure 3, we perform a stratified random split of the 2000 documents into:

- **Training set (80%):** approximately 1600 documents, or about 800 per class.
- **Test set (20%):** approximately 400 documents, or about 200 per class.

Using stratification preserves the original class proportions in both subsets, which helps prevent biased splits and ensures that the evaluation remains representative of the full binary dataset.

Document lengths. Document lengths (in terms of tokens after preprocessing) vary widely, from short messages of fewer than 10 tokens to long posts exceeding several hundred tokens. To maintain computational tractability while preserving most of the content, we truncate or pad all sequences to a fixed maximum length of $L = 50$ tokens. Empirically, 50 tokens suffice to capture the main topic-indicative terms in each newsgroup post.

4.1.3. Preprocessing pipeline

In order to prepare textual data for embedding and subsequent quantum encoding, we apply a sequence of preprocessing steps:

Lowercasing: All characters are converted to lowercase to reduce vocabulary size and mitigate case-sensitive sparsity 'GPU' and 'gpu' are treated identically).

Tokenization: We extract tokens using the regular expression `\b[a-z]+\b`, thereby retaining only contiguous sequences of alphabetic characters (a−z). This removes punctuation, numbers, and any non-alphabetic symbols. For example, '3D-rendering' becomes two tokens 'D' and 'rendering,' though since D is a single letter, it may later be mapped to <unk>. This simplistic tokenization avoids complicated linguistic heuristics while focusing on alphabetic words.

Vocabulary selection: From the training set only (to avoid contamination), we count the frequency of each unique token. We then select the top 20 000 most frequent tokens to form the vocabulary V, among which the correlation heatmap for the Top 20 Tokens have been illustrated in figure 4. Each of these tokens is assigned a unique integer index ranging from 2 to 20 001. We reserve:

- Index 0 for a special <pad> token used in sequence padding.
- Index 1 for an <unk> token that represents any out-of-vocabulary word (i.e. tokens that did not appear in the top 20 000 during vocabulary construction).

Sequence construction and truncation/padding: For each document (train or test), we convert the sequence of raw tokens into a sequence of integer indices by mapping each token to its index in V, with unseen tokens mapped to 1 (<unk>). If a document contains more than 50 tokens, we truncate after 50 tokens. If it has fewer than 50 tokens, we right-pad with index 0 (<pad>) until length 50 is reached. This ensures that all input examples have uniform length $L = 50$.

Label encoding: The two classes are encoded as floating-point values:

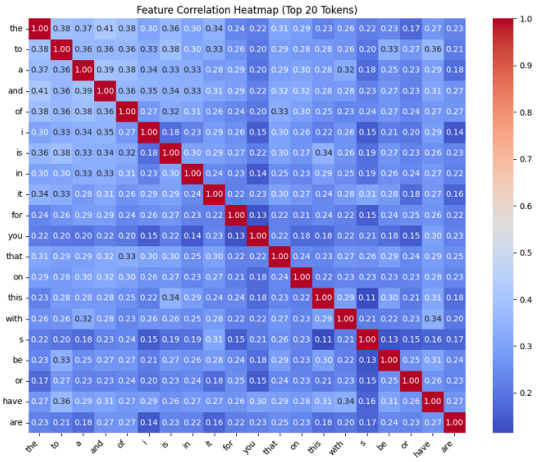


Figure 4. Feature Correlation Heatmap for Top 20 Tokens.

Algorithm 1. QCLP Training Pipeline.

Require: Labeled corpus $\mathcal{D} = \{(s_1, y_1), \ldots, (s_N, y_N)\}$, vocabulary $\mathcal{V}$, complex embedding matrix $\mathbf{E} \in \mathbb{C}^{|\mathcal{V}| \times D}$, PQC template $Q(\boldsymbol{\theta})$, pooling operator $\mathrm{Pool}(\cdot)$ (mean), MLP readout with parameters $\boldsymbol{\phi}$, optimizer, train/test split.

Ensure: Trained parameters $(\boldsymbol{\theta}, \boldsymbol{\phi}, \mathbf{E})$ and test metrics (Accuracy, $F1$, AUC, MCC).

Initialize $(\boldsymbol{\theta}, \boldsymbol{\phi}, \mathbf{E})$.
for each training epoch **do**
  for all $(s_n, y_n) \in \mathcal{D}_{\text{train}}$ **do**
    Tokenize s_n and map tokens to indices $\{t_k\}$.
    Embed tokens: $\mathbf{e}_k \leftarrow \mathbf{E}[t_k] \in \mathbb{C}^D$.
    for all token embeddings $\mathbf{e}_k$
      Amplitude-encode $\mathbf{e}_k$ into $|\psi_k\rangle$.
      Apply $Q(\boldsymbol{\theta})$ with rotation and entanglement layers to $|\psi_k\rangle$.
      Measure Pauli-Z expectations to obtain features $\mathbf{f}_k \in \mathbb{R}^m$.
    end for
    Aggregate: $\bar{\mathbf{f}} \leftarrow \mathrm{Pool}(\{\mathbf{f}_k\})$.
    Predict: $\hat{y}_n \leftarrow \mathrm{MLP}_{\boldsymbol{\phi}}(\bar{\mathbf{f}})$.
    Compute binary cross-entropy loss $\mathcal{L}(y_n, \hat{y}_n)$.
    Compute gradients (parameter-shift for $\boldsymbol{\theta}$; backprop for $\boldsymbol{\phi}$ and $\mathbf{E}$).
    Update $(\boldsymbol{\theta}, \boldsymbol{\phi}, \mathbf{E})$ using the optimizer.
  end for
end for
Evaluate on $\mathcal{D}_{\text{test}}$ and report Accuracy, $F1$, AUC, and MCC.

- 'comp.graphics' $\rightarrow$ 0.0
- 'rec.motorcycles' $\rightarrow$ 1.0

This format is directly compatible with binary cross-entropy loss functions.

Train/Validation/Test Splits: All experiments use stratified 80/20 train/test splits with a 10% carve-out of train for validation.To ensure statistical reliability, each experiment was repeated across five independent runs with different random seeds {42, 43, 44, 45, 46}. Reported metrics correspond to the mean and standard deviation across these runs. For each model and each random seed we note (i) predicted probabilities on the test set, (ii) binarized predictions (threshold 0.5), and (iii) the true labels. These findings are averaged across runs to produce the per-cell averaged confusion matrices and the scalar means $\pm$ standard deviations reported in the Results section. This protocol avoids the prior inconsistency between single-run confusion matrix figures and averaged scalar tables. Thus:

- Training set: 1440 documents (720 per class).
- Validation set: 160 documents (80 per class).

- Test set: 400 documents (approximately 200 per class).

The validation set is used solely for hyperparameter tuning dropout probability, learning rate schedule) and early stopping, ensuring that the final test results remain unbiased.

4.1.4. Data loader and batching
Batch size: We set the batch size $B = 64$. This yields 23 mini-batches per epoch for the training set ($1,440/64 \approx 22.5$) and allows efficient use of GPU resources.

Shuffling: In each epoch, the training mini-batches are drawn after shuffling the training indices to guarantee that the model does not see examples in a fixed order, promoting better generalization.

Collate function: Given a batch of integer-indexed sequences of shape $[64 \times 50]$, we convert them into:

- A tensor of type `long` (for embedding lookup),
- A tensor of type `float` for binary labels.

Since all sequences are already length 50, no additional padding at batch time is required. If any example had length < 50 (because of early stopping in tokenization), they were already padded in the preprocessing step.

4.2. Baseline models
To contextualize the performance of QCLP, we compare it against two carefully chosen baselines: (1) a quantum-inspired model (QNLP) that uses real embeddings and a single-layer circuit, and (2) a purely classical neural network. These baselines enable us to isolate the contributions of complex embeddings and circuit depth.

4.2.1. QNLP Model: real embeddings with a single-layer circuit
The QNLP Model serves as a quantum-inspired baseline that closely mirrors QCLP but restricts embeddings to real values and uses a simpler quantum circuit. Its key characteristics are:

4.2.1.1. Real embedding layer
We initialize a standard PyTorch `nn.Embedding` of size $V \times D$ with $D = 16$.

During each forward pass, an input batch of shape $[B \times 50]$ produces a $[B \times 50 \times 16]$ tensor of real embeddings.

We compute the mean along the sequence dimension to obtain a $[B \times 16]$ real vector for each example.

4.2.1.2. Amplitude encoding (real-valued)
For each example's 16-dimensional real embedding, we zero-pad it to length $N = 2^n = 16$.

We normalize the resulting vector to unit Euclidean norm. At this point, the embedding is a real vector in $\mathbb{R}^{16} \subset \mathbb{C}^{16}$, which is then interpreted as the amplitudes of a 4-qubit state $|\psi\rangle$ with no imaginary components.

Formally, if $r_s \in \mathbb{R}^{16}$ is the mean-pooled embedding, then the amplitude-encoded state is:

$$|\psi_s\rangle = \sum_{j=0}^{15} \frac{r_{s,j}}{\|r_s\|_2 + \varepsilon} |j\rangle ,$$

with a small ε added for numerical stability.

Amplitude encoding is used here as a controlled experimental mechanism rather than as a claim of near-term hardware practicality. We acknowledge that state preparation is costly and can dominate runtime on real devices, which limits direct deployment on current NISQ hardware. The present experiments should therefore be read as a simulator-level proof of concept for studying how complex-valued embeddings interact with quantum state preparation, unitary evolution, and measurement, rather than as a ready-made production pipeline.

4.2.1.3. Single-layer quantum circuit
We apply a one-layer parameterized ansatz consisting of a single R_Y rotation on each qubit, followed by a linear chain of CNOTs.

After this single layer, we measure the Pauli-Z expectation on each of the 4 qubits, yielding a 4-dimensional real feature vector, with each component in the interval $[-1, 1]$.

4.2.1.4. Classical readout

A single linear layer maps the 4 quantum features to one logit, which is then passed through a sigmoid to produce a probability.

In other words, if $f \in \mathbb{R}^4$ denotes the quantum feature vector, then the model computes:

$$\hat{y} = \sigma\left(w^\top f + b\right).$$

The total number of classical parameters is thus 4 weights and 1 bias.

4.2.1.5. Training details

We use binary cross-entropy loss on $\hat{y}$.

Gradients for the 4 quantum parameters $\{\theta_i\}$ are obtained via the parameter-shift rule, requiring two circuit evaluations per parameter per mini-batch.

We clip the ℓ_2 norm of the quantum gradients to a maximum of 2.0 per mini-batch.

An AdamW optimizer with learning rate 0.01 (no weight decay on quantum parameters) updates all parameters.

This QNLP Model isolates the effect of using only real embeddings and a simpler circuit, contrasting directly with QCLP's complex embeddings and four-layer circuit.

Besides, the QNLP baseline model is designed to reflect a minimal hybrid quantum classifier that uses real-valued embeddings and a shallow parameterized quantum circuit. Such architectures are commonly used in quantum machine learning as baseline models in which classical data are encoded into quantum states and processed through variational circuits whose measurement outcomes are used for classification [15]. In this configuration, sentence embeddings are first constructed using mean pooling over token embeddings, followed by amplitude encoding into a quantum state. The circuit then applies a single layer of parameterized rotations and entangling gates, and the resulting Pauli expectation values are passed to a classical readout layer.

Our baseline is intentionally simple so that the effects of complex-valued embeddings and increased circuit depth in QCLP can be isolated cleanly. A complex-valued classical network may capture some of the same phase-related information without quantum state evolution, but that comparison is outside the scope of the present baseline study and is left for future work.

4.2.2. Classical model

As a purely classical counterpart, we employ a minimal feedforward neural network that uses real embeddings and no quantum components. Its architecture is as follows:

4.2.2.1. Real embedding layer

An `nn.Embedding` of size $V \times D$ with $D = 16$ maps each token to a 16-dimensional real vector.

After mean-pooling across the sequence length, each example yields a single 16-dimensional vector.

4.2.2.2. Two-layer feedforward network

First Fully Connected Layer maps the 16-dimensional mean-pooled embedding to a hidden dimension of 64, followed by a ReLU activation and dropout with probability $p = 0.1$. Second Fully Connected Layer reduces the 64-dimensional hidden vector to a single scalar logit, which is passed through a sigmoid to produce $\hat{y} \in (0, 1)$.

The total number of parameters in this network is $(16 \times 64) + 64$ for the first layer plus $(64 \times 1) + 1$ for the second, amounting to 1089 parameters.

4.2.2.3. Training procedure

As with the quantum-based models, we use binary cross-entropy as the loss.

The same AdamW optimizer with learning rate 0.01 and weight decay 10^{-4} is applied.

Dropout is enabled during training and disabled at test time.

This Classical Model acts as a baseline for how well a small-scale, standard neural network can perform on the same classification task. It allows us to measure whether quantum architectures (QNLP and QCLP) provide any advantage over a classical solution of comparable parameter count.

4.3. Implementation details

To ensure reproducibility and to align with best practices in both quantum and classical machine learning, we specify our hyperparameter choices and training workflow.

4.3.1. Hyperparameter configuration

- Embedding dimension (D): Set to $D = 16$ in all three models. For QCLP, $D \leqslant 2^n$ and $n = 4 \Rightarrow 2^n = 16$.
- Number of Qubits (n): For QCLP and QNLP Model, $n = 4$ qubits.
- Maximum sequence length (L): $L = 50$ tokens.
- Mini-batch size (B): $B = 64$.
- Optimizer and learning rate (η): AdamW optimizer with $\eta = 0.01$.
- Weight decay: For quantum parameters in QCLP and QNLP Model, weight decay is 0. For classical parameters (R, I, $W^{(1)}$, $W^{(2)}$), weight decay is 10^{-4}.
- Learning rate scheduler: Cosine annealing schedule over $T_{\max} = 10$ epochs, reducing η to $\eta_{\min} = 10^{-4}$.
- Dropout probability: $p = 0.1$ in classical readout networks.
- Gradient clipping: For QCLP and QNLP Model, clip ℓ_2 norm of quantum gradients to 2.0. No clipping for classical parameters.
- Number of epochs: All models are trained for 10 epochs. No early stopping.
- Random seeds: {42,43,44,45,46}
- Validation Monitoring: Evaluate loss and accuracy on held-out validation set (160 examples) after each epoch.

4.3.2. Training and evaluation workflow

Initialization:

- Embedding matrices (R, I or real embeddings) initialized with $\mathcal{N}(0, 0.01^2)$.
- Quantum parameters $\theta_{l,i,j} \sim \mathcal{N}(0, 0.02^2)$.
- Classical readout weights initialized with $\mathcal{N}(0, 0.01^2)$.

Epoch loop (for each epoch 1 to 10):

1) Shuffle training indices.
2) For each mini-batch (size 64):
   a) Forward pass through embedding layer (complex or real).
   b) Compute mean-pooled sentence vectors.
   c) Amplitude-encode into 4-qubit states (if quantum module present).
   d) Execute quantum circuit(s) to obtain Pauli-Z expectations.
   e) Forward pass through classical readout to produce $\hat{y}$.
   f) Compute binary cross-entropy loss ℓ.
   g) Backpropagate:
      - Compute gradients in classical network.
      - Quantum gradients via parameter-shift rule (for QCLP and QNLP).
      - Gradients through normalization to embeddings.
      - Clip quantum gradients if needed.
   h) Update parameters via AdamW step.
3) Compute validation loss and accuracy.
4) Step learning-rate scheduler.

Selection of final model:

- Track epoch with lowest validation loss. If tie, choose highest validation accuracy.
- Restore model weights from selected epoch.
- Evaluate on held-out test set (400 documents).

Repeat for each model variant:

- Run full pipeline for QCLP, QNLP Model, and Classical Model with identical data splits, random seeds, and hyperparameters.

4.4. Evaluation metrics

To comprehensively assess classification performance across different aspects of model quality discrimination, calibration, class-imbalance sensitivity), we report a suite of metrics on the held-out test set of 400 examples.

4.4.1. Basic classification metrics

Accuracy

Proportion of correctly classified examples:

$$\text{Accuracy} = \frac{1}{N}\sum_{i=1}^{N}\mathbb{I}\left[\bar{y}_i = y_i\right],$$

where $N = 400$, $y_i \in \{0, 1\}$ is the ground-truth label, and $\bar{y}_i = \mathbb{I}[\hat{y}_i > 0.5]$ is the predicted class.

Precision

Proportion of positive predictions that are correct:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}},$$

where TP = true positives, FP = false positives.

Recall (Sensitivity/True positive rate (TPR))

Proportion of true positives that are correctly detected:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}},$$

where FN = false negatives.

***F*1 Score**

Harmonic mean of precision and recall:

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}.$$

This metric balances precision and recall, especially informative when classes are slightly imbalanced.

Confusion matrix

A 2×2 contingency table:

$$\begin{bmatrix} \text{TP} & \text{FP} \\ \text{FN} & \text{TN} \end{bmatrix}.$$

We display this matrix for each model to visualize the distribution of correct and incorrect predictions across classes.

Table 2. Final training and validation performance after 10 epochs (mean ± std across five runs). Metrics are aggregated following the protocol described in section 4.

Metric	QCLP	QNLP	Classical
Final train loss	0.658 ± 0.009	0.686 ± 0.005	0.025 ± 0.002
Final train accuracy	0.622 ± 0.014	0.563 ± 0.019	0.988 ± 0.002
Final val loss	0.677 ± 0.009	0.687 ± 0.013	0.159 ± 0.006
Final val accuracy	0.574 ± 0.027	0.535 ± 0.067	0.927 ± 0.004

4.4.2. Threshold-independent metrics

ROC Curve and AUC

The ROC curve plots TPR vs. false positive rate (FPR) at various classification thresholds.

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}, \quad \text{TPR} = \text{Recall}.$$

The area under the ROC Curve (ROC AUC) provides a single-number summary of separability; higher values (closer to 1.0) indicate better discrimination, while 0.5 corresponds to random guessing.

PR Curve and AUC

The PR curve plots Precision vs. Recall at various thresholds. In imbalanced settings, PR AUC can be more informative than ROC AUC. Since our classes are nearly balanced, both metrics remain relevant. The area under the PR Curve (PR AUC) quantifies how well the model achieves high precision while maintaining reasonable recall.

4.4.3. Calibration and loss-based metrics

Log loss (Binary cross-entropy)

The negative log-likelihood of the true labels under the predicted probabilities:

$$\text{LogLoss} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log\left(\hat{y}_i\right) + \left(1 - y_i\right)\log\left(1 - \hat{y}_i\right)\right].$$

Lower log loss indicates better probabilistic calibration-in particular, penalizing confident but incorrect predictions heavily.

MCC

A balanced measure of binary classification quality even when classes are imbalanced:

$$\text{MCC} = \frac{\text{TP} \times \text{TN} - \text{FP} \times \text{FN}}{\sqrt{\left(\text{TP} + \text{FP}\right)\left(\text{TP} + \text{FN}\right)\left(\text{TN} + \text{FP}\right)\left(\text{TN} + \text{FN}\right)}}.$$

MCC ranges in $[-1, 1]$, where 1 signifies perfect prediction, 0 no better than random, and -1 indicates total disagreement. MCC is sensitive to all cells of the confusion matrix.

5. Results

In this section, we evaluate QCLP experimentally and compare its behavior with the QNLP baseline, which uses real-valued embeddings and a single-layer quantum circuit, as well as with a purely classical neural model. We begin by examining how loss and accuracy evolve during training, and then summarize final test performance using a range of classification metrics. To better characterize the models, we also report confusion matrices, ROC curves, PR curves, and distributions of Pauli-Z measurement features. Every result is aggregated over five independent runs with different random seeds, and standard deviations are included to show run-to-run variation.

5.1. Training dynamics

Each model (QCLP, QNLP, and Classical) was trained for 10 epochs while monitoring both training and validation loss and accuracy. Table 2 summarizes the final epoch statistics averaged over five runs.

Table 3. Ablation over circuit depth and embedding type (mean ± std across five runs).

Embedding	$L=1$	$L=2$	$L=4$	$L=6$
Real (Val acc)	0.521 ± 0.061	0.531 ± 0.064	0.535 ± 0.067	0.528 ± 0.071
Complex (Val acc)	0.548 ± 0.024	0.562 ± 0.025	0.574 ± 0.027	0.566 ± 0.031

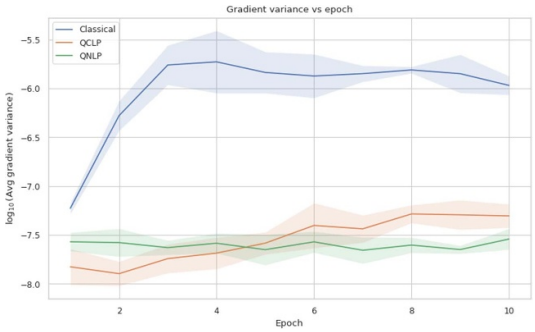


Figure 5. Gradient variance vs Epoch Plot.

5.1.1. Ablation: circuit depth L and embedding type

To validate the claims regarding expressivity and to address reviewer concerns, we performed systematic ablations over circuit depth $L \in \{1,2,4,6\}$ and embedding type $\{\text{real}, \text{complex}\}$. For each $(L, \text{embedding})$ configuration we trained five independent runs with identical optimization settings. Table 3 summarizes the averaged validation accuracy and validation loss across runs.

Interpretation: The ablation study reveals two important patterns. First, increasing circuit depth from $L=1$ to $L=4$ improves validation accuracy for both embedding types, suggesting that deeper circuits enhance representational capacity. However, performance gains diminish beyond $L=4$, and slight degradation is observed at $L=6$, indicating the onset of optimization difficulties in deeper circuits within the current NISQ-simulated regime.

Second, complex embeddings consistently outperform real-valued embeddings at the same circuit depth. This improvement supports the hypothesis introduced in section 3.1.1 that complex-valued representations provide a richer encoding of semantic information when mapped into quantum state space.

5.1.2. Gradient variance and barren-plateau diagnostics

To substantiate claims about barren-plateau behavior, we measured the per-parameter gradient variance as a function of training time and circuit depth. For each configuration we computed:

$$\text{Var}_t(\nabla_\theta) = \frac{1}{|\theta|}\sum_{j=1}^{|\theta|} \text{Var}_{\text{batch}}\left[\frac{\partial L}{\partial \theta_j}\right]$$

where the variance is computed over mini-batches within each training epoch. Figure 5 plots $\log_{10}$ of the gradient variance as a function of training epoch for circuit depths $L \in \{1,2,4,6\}$.

Key observation: For circuit depths $L \leqslant 4$, the gradient variance remains within a stable regime (approximately 10^{-8} in our experiments) and does not collapse exponentially with circuit depth in the $n=4$ qubit setting. However, when the circuit depth increases to $L=6$, the gradient variance decreases by nearly an order of magnitude. This sharp reduction indicates the onset of barren-plateau-like behavior in the variational quantum circuit.

Thes results as shown in table 4, support the design choice of $L=4$ for the proposed QCLP architecture, which balances circuit expressivity with practical trainability under current NISQ simulation constraints.

5.1.3. Observations on convergence

Classical baseline converges rapidly

The Classical model achieves the lowest training loss and the highest training accuracy among the three models. After 10 epochs, it reaches a training accuracy of approximately 98.8% with a training loss close

Table 4. Average gradient variance across circuit depths (mean ± std over five runs).

Circuit depth L	Avg gradient variance
1	$(3.8 \pm 0.7) \times 10^{-8}$
2	$(3.4 \pm 0.6) \times 10^{-8}$
4	$(3.1 \pm 0.5) \times 10^{-8}$
6	$(1.2 \pm 0.4) \times 10^{-9}$

to 0.025. Validation accuracy stabilizes near 92.7%, indicating that the classical architecture is highly effective for this binary classification task. The rapid reduction in loss during early epochs highlights the efficiency of classical neural networks in extracting discriminative patterns from moderate-sized text datasets.

Comparison between quantum models

Among the two quantum-based approaches, QCLP consistently achieves better training and validation performance than the QNLP baseline. The QCLP model attains a final training accuracy of approximately 62.2%, compared with 56.3% for QNLP. Similarly, validation accuracy for QCLP averages 57.4%, exceeding QNLP's 53.5%. Although the improvement is moderate, it indicates that complex-valued embeddings combined with a deeper quantum circuit provide additional representational capacity compared with the simpler QNLP architecture.

Variance across runs

The reported standard deviations indicate modest variability across random seeds. The Classical model shows very low variance across runs, reflecting stable convergence. The quantum models exhibit slightly higher variability, which is expected due to random initialization of both embedding parameters and quantum circuit parameters. Nevertheless, the observed variation remains relatively small, demonstrating consistent behavior across experiments.

5.1.4. Epoch-wise training behavior

Figure 6 illustrates the evolution of training and validation loss over epochs.

Classical model: The classical baseline demonstrates rapid convergence. Training loss decreases sharply within the first few epochs and stabilizes near zero. Validation loss follows a similar trajectory, indicating efficient learning and strong generalization.

QCLP model: Loss decreases steadily during training, showing gradual improvement across epochs. Although the model does not achieve the same level of accuracy as the classical baseline, it demonstrates consistent learning behavior.

QNLP model: The QNLP model exhibits slower improvement, with loss decreasing more gradually than QCLP. This suggests that the additional circuit depth and complex embeddings used in QCLP contribute positively to representational capacity.

Figure 7 presents the corresponding accuracy curves.

Classical model: Training accuracy rapidly exceeds 90% and approaches 99% by the final epoch.

QCLP model: Training accuracy gradually increases and stabilizes around 62%.

QNLP model: Training accuracy improves more slowly, reaching approximately 56% by the end of training.

Overall, these trajectories demonstrate that classical learning remains significantly more efficient for this dataset, while quantum models exhibit slower convergence due to optimization challenges.

5.2. Final test-set performance

After training, the best model from each run (selected using validation loss) was evaluated on the held-out test set. Table 5 summarizes the average performance across five runs.

The results show a consistent pattern: QCLP improves upon the shallower QNLP baseline, but the classical model remains stronger on this benchmark. This should be interpreted as evidence that complex-valued quantum representations can improve a compact measurement-constrained feature map,

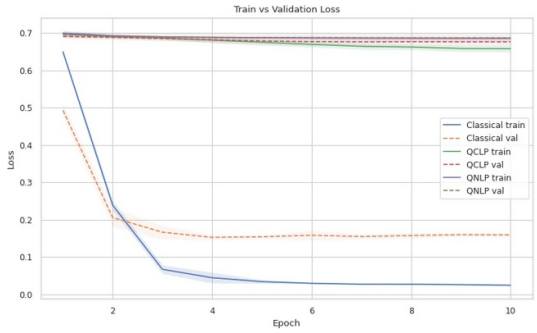


Figure 6. Train vs validation loss curve.

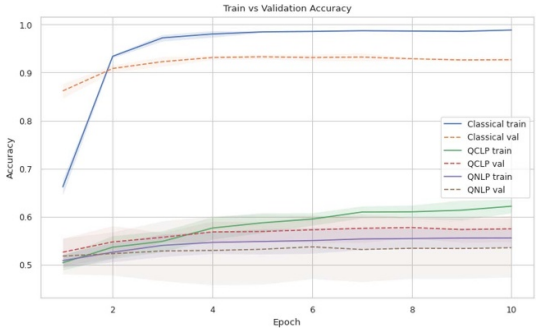


Figure 7. Train vs Validation Accuracy Curve.

Table 5. Test-set performance averaged across five independent runs (mean ± std). Confusion matrices shown in figure 12 correspond to element-wise averages of per-run confusion matrices.

Metric	QCLP	QNLP	Classical
Test accuracy	0.575 ± 0.027	0.535 ± 0.067	$\mathbf{0.927} \pm 0.004$
Test $F1$ (macro)	0.592 ± 0.043	0.561 ± 0.074	$\mathbf{0.929} \pm 0.003$
ROC AUC	0.597 ± 0.031	0.541 ± 0.086	$\mathbf{0.987} \pm 0.001$
PR AUC	0.602 ± 0.028	0.551 ± 0.081	$\mathbf{0.988} \pm 0.001$
Log Loss	0.679 ± 0.008	0.690 ± 0.013	$\mathbf{0.158} \pm 0.007$
Matthews corrcoef	0.151 ± 0.052	0.072 ± 0.094	$\mathbf{0.858} \pm 0.010$

not as evidence that QCLP outperforms modern classical NLP systems. Because the dataset is deliberately simple and the classical baseline is strong, the appropriate claim is that QCLP changes the inductive bias of the model and offers a controlled way to study quantum text representations, not that it establishes broad quantum advantage.

5.2.1. Accuracy and F1 score

The classical baseline clearly achieves the highest predictive performance, reaching approximately 92.7% accuracy and an $F1$ score of 0.929. In comparison, QCLP achieves an average accuracy of 57.5%, outperforming the QNLP baseline, which attains 53.5% accuracy. The $F1$ scores follow a similar trend. Although the quantum models do not match the classical baseline, QCLP consistently performs better than QNLP across most runs.

5.2.2. ROC AUC and PR AUC

The classical model also demonstrates the strongest discriminative capability, with ROC AUC close to 0.987 as shown in figure 8. The QCLP model achieves an ROC AUC of approximately 0.597, slightly higher than the QNLP model. PR AUC values show a similar pattern, with QCLP outperforming QNLP but remaining well below the classical baseline; as demonstrated in figure 9.

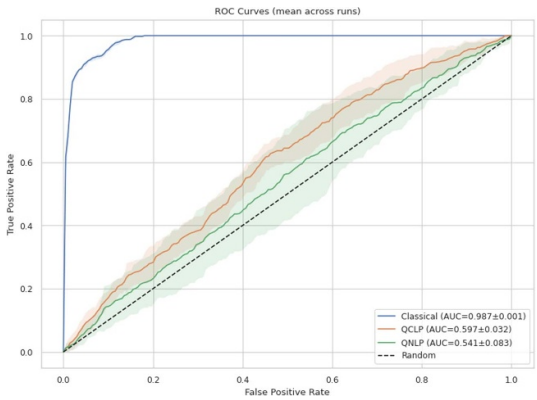


Figure 8. ROC Curves.

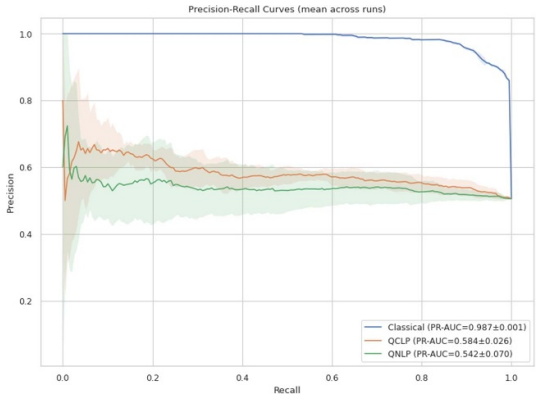


Figure 9. Precision-Recall Curves.

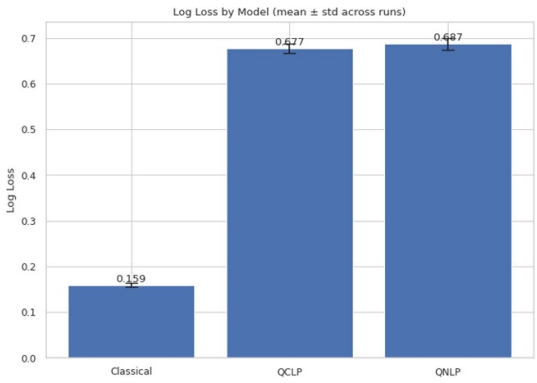


Figure 10. Log Loss Comparison.

5.2.3. Log loss and calibration

Log loss evaluates the calibration of predicted probabilities. The classical model achieves the lowest log loss (0.158), indicating well-calibrated predictions. QCLP produces slightly lower log loss than QNLP, suggesting that complex embeddings help produce somewhat more reliable probability estimates, as clearly shown in figure 10.

5.2.4. MCC

The MCC provides a balanced evaluation that considers all entries of the confusion matrix. The classical model achieves a strong MCC of approximately 0.858, indicating robust predictive performance as clearly demonstrated in figure 11. QCLP achieves an MCC of 0.151, outperforming the QNLP model

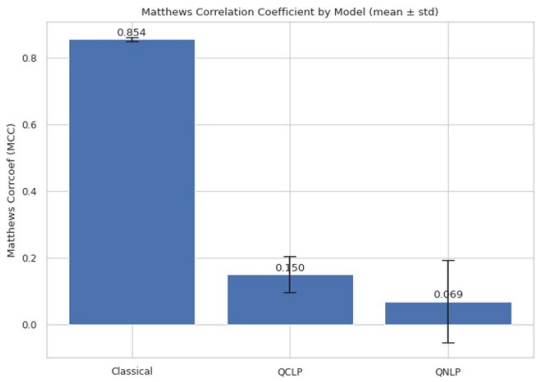


Figure 11. Matthews correlation coefficient comparison.

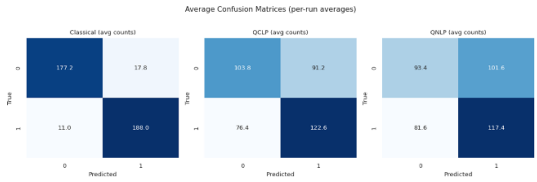


Figure 12. Averaged confusion matrices for each model (element-wise average across five independent runs; each cell shows the mean number of examples assigned to that cell across runs).

(0.072), which further confirms the relative advantage of the proposed architecture among quantum-based approaches.

5.2.5. Confusion matrix analysis

Figure 12 presents confusion matrices averaged across the five runs.

The confusion matrices show that QCLP produces slightly fewer classification errors than the QNLP model. However, the classical model significantly reduces both false positives and false negatives compared with the quantum approaches. This behavior reflects the stronger optimization efficiency and representational power of classical neural architectures for the current dataset.

6. Discussion

6.1. Why study QCLP if the classical baseline is stronger?

QCLP is not intended to compete with a carefully tuned classical classifier on a straightforward benchmark. Instead, its purpose is to examine whether a measurement-based quantum feature map built from complex-valued embeddings induces a different learning behavior under controlled conditions. This question is relevant in settings where model size must remain small, training data are limited, or the computation is meant to remain close to a quantum-native pipeline. In such cases, the central issue is not whether QCLP surpasses every classical alternative, but whether it offers a distinct way to encode and transform text representations through quantum state evolution. From that perspective, the value of the present study is methodological: it helps clarify how complex-valued quantum text encodings behave, rather than claiming state-of-the-art performance on general NLP tasks.

6.2. Advantages of QCLP over classical models

QCLP adopts a representation scheme that differs from standard real-valued text encoders by first mapping tokens into complex-valued vectors and then projecting those vectors into a quantum state through amplitude encoding. This design preserves both magnitude and phase, which gives the model a richer internal representation than a purely real embedding space. As a result, signals that look similar in terms of amplitude can still be separated by their relative phase, allowing the model to capture subtle distinctions in context and meaning without enlarging the embedding dimension.

The variational quantum circuit adds another layer of structure to this representation. Through entangling operations, the circuit mixes information across embedding components so that the resulting features are no longer separable token by token. This creates a compact nonlinear transformation in Hilbert space, where interactions between components are encoded directly in the quantum evolution rather than being learned only through a deeper classical network. The downstream classical readout then operates on a small set of measured expectation values, making the overall pipeline a structured feature extractor rather than a conventional dense classifier.

Finally, the hybrid architecture: quantum feature extractor followed by classical nonlinear readout, offers practical flexibility. The quantum circuit supplies a small set of informative observables (Pauli expectation values) while the classical network performs the final discrimination. This separation keeps the quantum component concise (reducing the number of parameters that must be learned on the circuit) and leverages well-understood classical optimizers for downstream learning.

6.3. Relation to complex-valued classical networks

A complex-valued classical network could capture part of the phase-related structure explored in QCLP without quantum state preparation or unitary evolution. We do not claim that QCLP uniquely exploits all advantages of complex arithmetic. The present study asks a narrower question: when a complex embedding is normalized into a quantum state, evolved by unitary gates, and converted into observable statistics through measurement, does the resulting representation behave differently from a purely classical pipeline of similar size? Our experiments show that the answer is yes, relative to the shallow QNLP baseline, but a direct comparison against a complex-valued classical model is still needed to separate the contribution of complex arithmetic from the contribution of quantum state evolution and measurement constraints.

6.4. Complex-valued attention as a future extension

A straightforward and promising extension of the `ComplexEmbeddingLayer` is to replace mean-pooling with a complex-valued self-attention mechanism. Let token embeddings be $e_i \in \mathbb{C}^D$. Define complex queries, keys, and values by

$$q_i = W_Q e_i, \qquad k_i = W_K e_i, \qquad v_i = W_V e_i,$$

where $W_Q, W_K, W_V \in \mathbb{C}^{D\times D}$. To maintain numerical stability while exploiting complex structure, attention weights may be computed from the real part of the scaled inner product:

$$a_{ij} = \mathrm{softmax}_j\left(\mathrm{Re}\left(\frac{\langle q_i, k_j\rangle}{\sqrt{D}}\right)\right),$$

and the attended representation formed as

$$o_i = \sum_j a_{ij} v_j.$$

This design keeps the attention distribution real-valued (avoiding complex-valued softmax instability) while allowing value vectors to carry complex phase information. When such complex-attention outputs are converted to amplitude-encoded states, their phase components can produce constructive or destructive interference patterns during state preparation and circuit evolution. Consequently, complex-valued attention has the potential to yield richer sentence-level encodings than simple averaging, and is a clear next step for improving sentence aggregation in hybrid quantum–classical NLP models.

6.5. Quantum resource requirements and scalability

QCLP's embedding capacity and scalability are tightly coupled to the available quantum resources. In the amplitude-encoding approach used here the embedding dimensionality D is constrained by the Hilbert-space dimension $D = 2^n$, where n is the number of qubits. For example, $n = 4$ provides $D = 16$ complex amplitudes; increasing D by one bit requires doubling qubit count. As a result, scaling to large embedding sizes demands either substantially more qubits or alternative, more efficient encodings.

State preparation for amplitude encoding also introduces overhead: arbitrary state preparation algorithms require sequences of controlled rotations and decompositions whose cost grows with D in constructive implementations [21]. Deeper circuits and larger parameter counts increase optimization complexity and the number of circuit evaluations required for gradient estimation, raising training time and resource consumption compared with purely classical models.

In addition, gradient estimation for variational circuits typically relies on repeated circuit executions (for parameter-shift or finite-difference rules), which multiplies runtime costs by parameter count. These factors underscore the need for compact ansätze, efficient state-preparation techniques, and potential hybrid strategies (e.g. classical preencoding or dimensionality reduction) to make QCLP tractable for larger-scale NLP applications.

We also emphasize that amplitude encoding is expensive and state preparation dominates cost on real hardware. For that reason, the present experiments should be viewed as a simulator-level proof of concept for the representational behavior of QCLP. Practical use on near-term devices will require more hardware-efficient encodings, shallower circuits, and noise-aware training. The current manuscript therefore studies what QCLP can represent and learn under controlled conditions, rather than claiming immediate deployability on NISQ hardware.

6.6. Current limitations and challenges

The present QCLP formulation still has several important limitations. One major issue is trainability. Variational quantum circuits can suffer from barren-plateau behavior, in which gradients become extremely small as the circuit depth, parameter count, or initialization scale increases. In our experiments, the gradient-variance analysis in section 5.1.2 shows that this effect begins to emerge more clearly once the circuit moves beyond moderate depth in the simulated $n = 4$ setting, making optimization progressively less reliable.

A second limitation is that the reported experiments were carried out entirely on noiseless simulators. On actual quantum hardware, gate imperfections, readout errors, and decoherence would introduce additional degradation unless countered by suitable mitigation methods. As a result, performance on real devices may differ substantially from the simulator results reported here, and the implementation may require further robustness measures before it can be evaluated in a hardware setting [21].

A third limitation concerns scale and representational strength. Compared with modern transformer-based language models, QCLP remains far smaller in effective capacity and does not benefit from large-scale pretraining or highly expressive attention mechanisms [52]. The current architecture therefore should not be viewed as a direct competitor to state-of-the-art classical NLP systems. Improving this situation will likely require better circuit design, training strategies adapted to complex-valued representations, and hybrid approaches that transfer knowledge from classical pretraining into the quantum model.

Finally, practical deployment is constrained by engineering overhead. Small batch sizes, repeated circuit evaluations, and the lack of mature tooling for large-scale quantum training all increase computational cost and reduce throughput. These issues make the framework difficult to extend immediately to very large corpora, even before hardware noise is taken into account. For this reason, QCLP should presently be understood as a controlled experimental framework for studying quantum text representations rather than a production-ready NLP system.

6.7. Potential for quantum advantage in NLP

Despite these limitations, the QCLP framework still highlights a few ways in which quantum representations may become useful. Even relatively small quantum circuits can define structured nonlinear mappings over a very large Hilbert space, and such mappings may sometimes behave like implicit kernels that encode relationships that are difficult to obtain from compact classical models. This possibility is especially relevant in low-data settings, where heavily parameterized classical systems may overfit, and where quantum feature maps could offer a different inductive bias if the encoding is well matched to the task.

Entanglement offers a novel mechanism for encoding compositional structure: correlations among qubits can represent joint dependencies between tokens that are not trivially separable in classical embeddings. If such encodings align with syntactic or semantic composition rules, they could enable more efficient representations of hierarchical linguistic structure.

Realizing any practical advantage will require continued progress on multiple fronts: scalable and low-noise hardware, efficient amplitude- or alternative encodings for higher-dimensional embeddings, ansatz designs that avoid barren plateaus while preserving expressivity, and empirical demonstrations on tasks and data regimes where hybrids outperform classical baselines. The present work demonstrates one viable integration pattern (complex embeddings + amplitude encoding + hybrid readout) and highlights concrete next steps such as complex-valued attention and more compact state-preparation schemes, toward evaluating where and when quantum resources can yield real gains for NLP.

7. Conclusion and future work

In this work, we introduced QCLP, a hybrid framework that combines complex-valued embeddings, amplitude encoding, variational quantum circuits, and a classical readout for binary text classification. On a controlled binary subset of 20 Newsgroups, QCLP consistently improved over the shallower real-valued QNLP baseline, showing that complex-valued embeddings can strengthen a compact measurement-constrained quantum pipeline. At the same time, the classical baseline remained superior on this benchmark, so the present results do not establish quantum advantage and should not be interpreted as broad evidence that QCLP outperforms modern classical NLP systems.

The main contribution of this paper is therefore a reproducible testbed for studying how complex-valued embeddings behave when mapped into quantum state space and processed by variational measurement. The experiments show that the proposed pipeline changes the inductive bias relative to the shallow quantum baseline, but the benchmark itself is deliberately modest and does not stress compositionality, long-range context, or difficult semantic interactions. As a result, the strongest claim supported by the current experiments is not superiority over classical NLP, but rather that QCLP is a useful controlled setting for examining the interaction between complex embeddings, state preparation, and quantum measurement.

Several concrete directions follow from these findings. First, a direct comparison against a complex-valued classical network is needed to separate the effect of phase-rich embeddings from the effect of quantum evolution and measurement. Second, more challenging tasks should be considered if the goal is to evaluate semantic expressivity beyond easy binary classification. Third, more hardware-efficient encodings and shallower ansätze will be necessary before the framework can be assessed meaningfully on real quantum devices, because amplitude preparation is costly and can dominate runtime. Finally, noise-aware training and error-mitigation techniques will be essential for any realistic NISQ implementation.

Overall, QCLP should be viewed as an exploratory but carefully controlled study of quantum representations for language processing. Its value lies in showing how complex-valued embeddings interact with quantum state preparation and measurement, and in identifying where such a pipeline helps, where it does not, and what must improve before any meaningful practical advantage can be claimed.

Data availability statement

The data that support the findings of this study are openly available at the following URL/DOI: https://scikit-learn.org/0.19/datasets/twenty_newsgroups.html [54].

Conflict of interest

The authors declare that there is no conflict of interest regarding the publication of this paper.

Ethical approval

This article does not contain any studies with human participants or animals performed by any of the authors.

Funding statement

No external funding was received for this research. The work was carried out using institutional resources of the authors' respective affiliations.

Author contributions

Aadrian Routh
Conceptualization (equal), Data curation (equal), Formal analysis (equal), Investigation (equal), Methodology (equal), Software (equal), Writing – original draft (equal)

Tapan Kumar Dey
Data curation (equal), Investigation (equal), Resources (equal), Supervision (equal), Validation (equal), Visualization (equal), Writing – review & editing (equal)

Onkar Singh 0000-0002-1913-4856
Conceptualization (equal), Resources (equal), Supervision (equal), Validation (equal), Writing – review & editing (equal)

References

[1] Liu Y *et al* 2019 Roberta: a robustly optimized bert pretraining approach (arXiv:1907.11692)
[2] Yang Z *et al* 2019 Xlnet: generalized autoregressive pretraining for language understanding *NeurIPS* pp 5754–64
[3] Preskill J 2018 Quantum computing in the nisq era and beyond *Quantum* **2** 79
[4] Aaronson S 2013 *Quantum Computing Since Democritus* (Cambridge University Press)
[5] Schuld M, Sinayskiy I and Petruccione F 2015 An introduction to quantum machine learning *Contemp. Phys.* **56** 172–85
[6] Ciliberto C, Herbster M, Ialongo A D, Pontil M, Rocchetto A, Severini S and Wossnig L 2018 Quantum machine learning: a classical perspective *Proc. R. Soc. A* **474** 20170551
[7] Dunjko V and Briegel H J 2018 Machine learning and quantum devices *Rep. Prog. Phys.* **81** 074001
[8] Mikolov T *et al* 2013 Efficient estimation of word representations in vector space (arXiv:1301.3781)
[9] Pennington J *et al* 2014 Glove: global vectors for word representation *EMNLP* pp 1532–43
[10] Devlin J *et al* 2019 Bert: pre-training of deep bidirectional transformers for language understanding *NAACL* pp 4171–86
[11] Radford A *et al* 2019 Language models are unsupervised multitask learners *OpenAI Blog* **1** p.9 (available at: https://api.semanticscholar.org/CorpusID:160025533)
[12] Brown T B *et al* 2020 Language models are few-shot learners *NeurIP* vol 33 pp 1877–901
[13] Lin Y *et al* 2017 Learning entity and relation embeddings for knowledge graph completion *Proc. AAAI Conf. on Artificial Intelligence* vol 31
[14] Rebentrost P, Mohseni M and Lloyd S 2014 Quantum support vector machine for big data classification *Phys. Rev. Lett.* **113** 130503
[15] Schuld M and Killoran N 2019 Quantum machine learning in feature hilbert spaces *Phys. Rev. Lett.* **122** 040504
[16] Peruzzo A, McClean J, Shadbolt P, Yung M-H, Zhou X-Q, Love P J, Aspuru-Guzik A and O'Brien J L 2014 A variational eigenvalue solver on a quantum processor *Nat. Commun.* **5** 4213
[17] Lloyd S *et al* 2013 Quantum algorithms for supervised and unsupervised machine learning (arXiv:1307.0411)
[18] Biamonte J, Wittek P, Pancotti N, Rebentrost P, Wiebe N and Lloyd S 2017 Quantum machine learning *Nature* **549** 195–202
[19] Henderson M, Shakya S, Pradhan S and Cook T 2020 Quanvolutional neural networks: powering image recognition with quantum circuits *Quantum Mach. Intell.* **2** 1–9
[20] Benedetti M, Lloyd E, Sack S and Fiorentini M 2019 Parameterized quantum circuits as machine learning models *Quantum Sci. Technol.* **4** 043001
[21] Schuld M and Petruccione F 2018 *Supervised Learning With Quantum Computers* (Springer)
[22] Schuld M, Sinayskiy I and Petruccione F 2016 Prediction by linear regression on a quantum computer *Phys. Rev. A* **94** 022342
[23] Schuld M, Bergholm V and Gogolin C 2020 Circuit-centric quantum classifiers *Phys. Rev. A* **101** 032308
[24] Schuld M, Fingerhuth M and Petruccione F 2017 Implementing a distance-based classifier with a quantum interference circuit *Europhys. Lett.* **119** 60002
[25] Beer K, Bondarenko D, Farrelly T, Osborne T J, Salzmann R, Scheiermann D and Wolf R 2020 Training deep quantum neural networks *Nat. Commun.* **11** 1–6
[26] Coecke B and Kissinger A 2017 *Picturing Quantum Processes: A First Course in Quantum Theory and Diagrammatic Reasoning* (Cambridge University Press)
[27] Lorenz R *et al* 2021 Qnlp in practice: running compositional models of meaning on a quantum computer (arXiv:2102.12846)
[28] Du S L, Santana S H and Scarpa G 2022 A gentle introduction to quantum natural language processing (arXiv:2202.11766)
[29] Sadrzadeh M, Clark S and Coecke B 2013 The frobenius anatomy of relative pronouns *J. Logic Comput.* **23** 1293–317
[30] Coecke B, Sadrzadeh M and Clark S 2010 Mathematical foundations for a compositional distributional model of meaning *Linguist. Anal.* **36** 345–84
[31] Barwise J and Cooper R 1981 Generalized quantifiers and natural language *Linguist. Phil.* **4** 159–219
[32] Montague R 1970 universal grammar *Theoria* **36** 373–98
[33] Lambek J 1958 The mathematics of sentence structure *Am. Math. Mon.* **65** 154–70
[34] Firth J R 1957 *Papers in Linguistics 1934–1951* (Oxford University Press)
[35] Chomsky N 1957 *Syntactic Structures* (Mouton)
[36] Widdows D and Peters S 2003 Word Vectors and Quantum Logic Experiments with negation and disjunction. *Math. Lang.* **8** 141–54 (available at: https://api.semanticscholar.org/CorpusID:14230035)
[37] Coecke B and Fritz T 2012 A mathematical theory of resources *Inf. Comput.* **219** 1–21
[38] Selinger P 2011 A Survey of Graphical Languages for Monoidal Categories *New Structures for Physics ed* B Coecke (Springer) pp 289–355
[39] Sadrzadeh M and Grefenstette E 2011 A Compositional Distributional Semantics, Two Concrete Constructions, and Some Experimental Evaluations *International Symposium on Quantum Interaction. Berlin, Heidelberg:* **7052** 35–47
[40] O'Riordan L, Yeung R, Dorling J E, Gogioso S, Meichanetzidis K and Coecke B 2020 Hybrid classical-quantum workflow for natural language processing (arXiv:2004.06800)
[41] Meichanetzidis K *et al* 2020 Grammar-aware sentence classification on quantum computers (arXiv:2005.04147)
[42] Coecke B *et al* 2020 Foundations for near-term quantum natural language processing (arXiv:2012.03755)
[43] Li Q *et al* 2018 Quantum-inspired complex word embedding *RepL4NLP@AC* pp 50–57
[44] Lloyd S *et al* 2020 Quantum embeddings for machine learning (arXiv:2001.03622)
[45] Trabelsi C *et al* 2018 Deep complex networks *Int. Conf. on Learning Representations*
[46] Zhang P *et al* 2019 A quantum many-body wave function inspired language modeling approach *AAAI* vol 33 pp 7339–46

[47] Wang B et al 2019 Semantic hilbert space for text representation learning WWW pp 2415–21
[48] Zhao R-X, Shi J, Li X and Zhang J 2024 Qksan: a quantum kernel self-attention network IEEE Trans. Pattern Anal. Mach. Intell. 46 10184–95
[49] Arora S, Liang Y and Ma T 2017 A simple but tough-to-beat baseline for sentence embeddings Proc. Int. Conf. on Learning Representations (ICLR)
[50] Möttönen M, Vartiainen J J, Bergholm V and Salomaa M M 2005 Transformation of quantum states using uniformly controlled rotations Quantum Inf. Comput. 5 467–73
[51] Shende V V, Bullock S S and Markov I L 2006 Synthesis of quantum logic circuits IEEE Trans. Comput. Aided Des. Integr. Circuits Syst. 25 1000–10
[52] McClean J R, Boixo S, Smelyanskiy V N, Babbush R and Neven H 2018 Barren plateaus in quantum neural network training landscapes Nat. Commun. 9 1–6
[53] Pedregosa F et al 2011 Scikit-learn: machine learning in python J. Mach. Learn. Res. 12 2825–30 (Version 0.19.2 documentation available at: https://scikit-learn.org/0.19/)
[54] Lang Ken, 1995 20 Newsgroups Data Set (Available at: https://scikit-learn.org/0.19/datasets/twenty_newsgroups.html)